

ElevatorControl

Generated by Doxygen 1.16.1

1 Topic Index	1
1.1 Topics	1
2 Data Structure Index	3
2.1 Data Structures	3
3 File Index	5
3.1 File List	5
4 Topic Documentation	7
4.1 Application Layer	7
4.1.1 Detailed Description	7
4.1.2 CallHandler	7
4.1.2.1 Detailed Description	8
4.1.2.2 Function Documentation	8
4.1.2.3 CallHandler Global Config	8
4.1.3 ElevatorController	9
4.1.3.1 Detailed Description	10
4.1.3.2 Function Documentation	10
4.1.3.3 ElevatorController Local Config	13
4.1.3.4 ElevatorController Global Config	19
4.1.4 LEDController	20
4.1.4.1 Detailed Description	20
4.1.4.2 Function Documentation	20
4.1.4.3 LEDController Global Config	22
4.1.5 Main	26
4.1.5.1 Detailed Description	27
4.1.5.2 Function Documentation	27
4.1.6 Menu	27
4.1.6.1 Detailed Description	28
4.1.6.2 Function Documentation	28
4.1.6.3 Menu Global Config	29
4.1.7 MotionController	30
4.1.7.1 Detailed Description	31
4.1.7.2 Function Documentation	31
4.1.7.3 MotionController Global Config	31
5 Data Structure Documentation	33
5.1 CallDioConfig_t Struct Reference	33
5.2 Elevator_t Struct Reference	33
5.2.1 Detailed Description	34
5.2.2 Field Documentation	34
5.2.2.1 aenuFloorCalls	34
5.2.2.2 bEmergencyStop	34

5.2.2.3	enumCollDir	34
5.2.2.4	enumDirection	35
5.2.2.5	enumDoorState	35
5.2.2.6	enumDoorType	35
5.2.2.7	enumOperatingMode	35
5.2.2.8	u16DoorTimer	35
5.2.2.9	u8CabinPwd	35
5.2.2.10	u8CamFailCnt	35
5.2.2.11	u8CamTimer	35
5.2.2.12	u8CollectionDir	36
5.2.2.13	u8CurrentFloor	36
5.2.2.14	u8DestinationFloor	36
5.2.2.15	u8FastTimer	36
5.2.2.16	u8FloorCount	36
5.2.2.17	u8LightTimer	36
5.2.2.18	u8MntSpeed	36
5.2.2.19	u8ParkFloor	36
5.2.2.20	u8PhsSeq	37
5.2.2.21	u8SlowTimer	37
5.2.2.22	u8StopTimer	37
5.3	FloorLED_t Struct Reference	37
5.3.1	Detailed Description	37
5.3.2	Field Documentation	37
5.3.2.1	blsActive	37
5.3.2.2	bLEDState	38
5.3.2.3	enumLEDCallType	38
5.3.2.4	u16BlinkTimer	38
5.4	LEDConfig_t Struct Reference	38
5.4.1	Detailed Description	38
5.4.2	Field Documentation	38
5.4.2.1	enumPattern	38
5.4.2.2	u16OffTime	39
5.4.2.3	u16OnTime	39
5.4.2.4	u8LedId	39
5.5	LEDHWConfig_t Struct Reference	39
5.5.1	Detailed Description	39
5.5.2	Field Documentation	39
5.5.2.1	u8BitPos	39
5.5.2.2	u8LatchIndex	40
5.6	LEDStatus_t Struct Reference	40
5.7	MenuItem_t Struct Reference	40
5.7.1	Detailed Description	40

5.7.2 Field Documentation	41
5.7.2.1 eepromAddress	41
5.7.2.2 label	41
5.7.2.3 maxValu	41
5.7.2.4 minValu	41
5.7.2.5 value	41
6 File Documentation	43
6.1 CallHandler_lcfg.h	43
6.2 CallHandler/interfaces/CallHandler.h File Reference	44
6.3 CallHandler.h	44
6.4 CallHandler/interfaces/CallHandler_gcfg.h File Reference	45
6.4.1 Detailed Description	45
6.5 CallHandler_gcfg.h	45
6.6 ElevatorController/includes/ElevatorController_lcfg.h File Reference	46
6.6.1 Detailed Description	48
6.7 ElevatorController_lcfg.h	48
6.8 ElevatorController/interfaces/ElevatorController.h File Reference	50
6.8.1 Detailed Description	51
6.9 ElevatorController.h	51
6.10 ElevatorController/interfaces/ElevatorController_gcfg.h File Reference	52
6.10.1 Detailed Description	53
6.10.2 Enumeration Type Documentation	53
6.10.2.1 CollDir_t	53
6.10.2.2 DoorState_t	53
6.10.2.3 DoorType_t	54
6.10.2.4 MoveDirection_t	54
6.10.2.5 OperatingMode_t	54
6.11 ElevatorController_gcfg.h	55
6.12 LEDController_lcfg.h	56
6.13 LEDController/interfaces/LEDController.h File Reference	57
6.13.1 Detailed Description	57
6.14 LEDController.h	57
6.15 LEDController/interfaces/LEDController_gcfg.h File Reference	58
6.15.1 Detailed Description	60
6.16 LEDController_gcfg.h	60
6.17 main.c File Reference	61
6.17.1 Detailed Description	61
6.18 Menu_lcfg.h	62
6.19 Menu/interfaces/Menu.h File Reference	62
6.19.1 Detailed Description	63
6.20 Menu.h	63

6.21 Menu/interfaces/Menu_gcfcg.h File Reference	64
6.21.1 Detailed Description	64
6.22 Menu_gcfcg.h	65
6.23 MotionController_lcfg.h	66
6.24 MotionController/interfaces/MotionController.h File Reference	66
6.24.1 Detailed Description	66
6.25 MotionController.h	67
6.26 MotionController/interfaces/MotionController_gcfcg.h File Reference	67
6.26.1 Detailed Description	68
6.27 MotionController_gcfcg.h	68
Index	69

Chapter 1

Topic Index

1.1 Topics

Here is a list of all topics with brief descriptions:

Application Layer	7
CallHandler	7
CallHandler Global Config	8
ElevatorController	9
ElevatorController Local Config	13
ElevatorController Global Config	19
LEDController	20
LEDController Global Config	22
Main	26
Menu	27
Menu Global Config	29
MotionController	30
MotionController Global Config	31

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

CallDioConfig_t	33
Elevator_t	
Holds the full runtime state of the elevator	33
FloorLED_t	
Runtime state of a single floor LED	37
LEDConfig_t	
Configuration entry for a single LED pattern	38
LEDHWConfig_t	
Maps a logical LED index to its physical latch and bit position	39
LEDStatus_t	40
MenuItem_t	
Data record for a single configurable menu item	40

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

main.c	Application entry point for the HexaMix Elevator Control system	61
CallHandler/includes/ CallHandler_lcfg.h		43
CallHandler/interfaces/ CallHandler.h	Public interface for the CallHandler module	44
CallHandler/interfaces/ CallHandler_gcfcg.h	Global configuration types for the CallHandler module	45
ElevatorController/includes/ ElevatorController_lcfg.h	Local (private) configuration constants for the ElevatorController module	46
ElevatorController/interfaces/ ElevatorController.h	Public interface for the ElevatorController module	50
ElevatorController/interfaces/ ElevatorController_gcfcg.h	Global configuration types for the ElevatorController module	52
LEDController/includes/ LEDController_lcfg.h		56
LEDController/interfaces/ LEDController.h	Public interface for the LEDController module	57
LEDController/interfaces/ LEDController_gcfcg.h	Global configuration types and hardware mappings for the LEDController module	58
Menu/includes/ Menu_lcfg.h		62
Menu/interfaces/ Menu.h	Public interface for the Menu module	62
Menu/interfaces/ Menu_gcfcg.h	Global configuration types for the Menu module	64
MotionController/includes/ MotionController_lcfg.h		66
MotionController/interfaces/ MotionController.h	Public interface for the MotionController module	66
MotionController/interfaces/ MotionController_gcfcg.h	Global configuration types for the MotionController module	67

Chapter 4

Topic Documentation

4.1 Application Layer

Collaboration diagram for Application Layer:

Topics

- [CallHandler](#)
- [ElevatorController](#)
- [LEDController](#)
- [Main](#)
- [Menu](#)
- [MotionController](#)

4.1.1 Detailed Description

4.1.2 CallHandler

Collaboration diagram for CallHandler:

Topics

- [CallHandler Global Config](#)

Functions

- void [CallHandler_vidInit](#) ([Elevator_t](#) *pstrElevator)
Initializes the CallHandler module.
- void [CallHandler_vidGetCall](#) (void)
Scans all call inputs and registers active calls.

4.1.2.1 Detailed Description

4.1.2.2 Function Documentation

4.1.2.2.1 CallHandler_vidGetCall()

```
void CallHandler_vidGetCall (
    void )
```

Scans all call inputs and registers active calls.

Reads each floor button (internal and external) and updates the [aenuFloorCalls](#) array accordingly. Should be called periodically from the main operation loop.

Scans all call inputs and registers active calls.

Parameters

<i>pstrCall</i>	struct holding call info.
-----------------	---------------------------

Here is the call graph for this function: Here is the caller graph for this function:

4.1.2.2.2 CallHandler_vidInit()

```
void CallHandler_vidInit (
    Elevator_t * pstrElevator)
```

Initializes the CallHandler module.

Must be called once during system startup before any call scanning is performed. Stores a reference to the shared elevator state.

Parameters

<i>pstrElevator</i>	Pointer to the shared Elevator_t state structure.
---------------------	---

Initializes the CallHandler module. Here is the caller graph for this function:

4.1.2.3 CallHandler Global Config

Collaboration diagram for CallHandler Global Config:

Enumerations

- enum [CallType_t](#) { [CALL_NONE](#) , [CALL_INTERNAL](#) , [CALL_EXTERNAL](#) }
Classifies the type of a floor call.

4.1.2.3.1 Detailed Description

4.1.2.3.2 Enumeration Type Documentation

4.1.2.3.2.1 CallType_t

enum [CallType_t](#)

Classifies the type of a floor call.

Enumerator

CALL_NONE	No active call registered for this floor.
CALL_INTERNAL	Call originated from inside the cabin (cabin panel button).
CALL_EXTERNAL	Call originated from a landing panel button.

4.1.3 ElevatorController

Collaboration diagram for ElevatorController:

Topics

- [ElevatorController Local Config](#)
- [ElevatorController Global Config](#)

Functions

- void **ElevatorController_Init** (void)
This function initializes and starts the operation loop of the elevator controller.
- void [ElevatorController_vidOperationLoop](#) (void)
Main periodic operation loop of the elevator controller.
- void [ElevatorController_vidProgrammingLoop](#) (void)
Programming loop of the elevator controller.
- uint8_t [ElevatorController_u8GetCurrentFloor](#) (void)
Retrieves the current floor position of the elevator.
- uint8_t [ElevatorController_u8GetSelectorCnt](#) (void)
Retrieves the selector pole count accumulated since the last reset.
- void [ElevatorController_vidStartLightTimer](#) (void)
Starts (or restarts) the cabin light auto-off timer.
- void [elevator_hal_vidInit](#) (void)
Initializes the HAL layer used by the elevator controller.
- void **elevator_hal_vidMotor_up** (void)
Commands the motor to run in the upward direction.
- void **elevator_hal_vidMotor_down** (void)
Commands the motor to run in the downward direction.
- void **elevator_hal_vidMotor_stop** (void)
Commands the motor to stop.
- void **elevator_hal_vidDoor_open** (void)
Commands the door actuator to open.

- void **elevator_hal_vidDoor_close** (void)
Commands the door actuator to close.
- void **elevator_hal_vidDisplay_floor** (uint8_t u8Floor)
Updates the floor indicator display with the given floor number.
- void **elevator_hal_vidDisplay_status** (const uint8_t *cpu8Status)
Updates the status display with a text string.
- void **elevator_hal_vidTimer_start** ()
Starts the HAL millisecond timer.
- void **elevator_hal_vidTimer_stop** (void)
Stops the HAL millisecond timer.
- uint16_t **elevator_hal_u16Get_time_ms** (void)
Returns the elapsed time in milliseconds since the timer was started.
- void **ElevatorController_vidSplashScreen** (void)
Displays the splash screen on the LCD at startup.
- **OperatingMode_t** **ElevatorController_enuGetMode** (void)
Returns the current operating mode of the elevator controller.

4.1.3.1 Detailed Description

4.1.3.2 Function Documentation

4.1.3.2.1 elevator_hal_u16Get_time_ms()

```
uint16_t elevator_hal_u16Get_time_ms (
    void )
```

Returns the elapsed time in milliseconds since the timer was started.

Returns

uint16_t Elapsed time in milliseconds.

4.1.3.2.2 elevator_hal_vidDisplay_floor()

```
void elevator_hal_vidDisplay_floor (
    uint8_t u8Floor)
```

Updates the floor indicator display with the given floor number.

Parameters

<i>u8Floor</i>	Floor number to show on the display (zero-based index).
----------------	---

4.1.3.2.3 elevator_hal_vidDisplay_status()

```
void elevator_hal_vidDisplay_status (
    const uint8_t * cpu8Status)
```

Updates the status display with a text string.

Parameters

<i>cpu8Status</i>	Pointer to a null-terminated status string.
-------------------	---

4.1.3.2.4 elevator_hal_vidInit()

```
void elevator_hal_vidInit (
    void )
```

Initializes the HAL layer used by the elevator controller.

Configures GPIO, timers, and peripheral drivers required by the elevator hardware abstraction layer. Here is the call graph for this function: Here is the caller graph for this function:

4.1.3.2.5 ElevatorController_enuGetMode()

```
OperatingMode_t ElevatorController_enuGetMode (
    void )
```

Returns the current operating mode of the elevator controller.

Returns

[OperatingMode_t](#) Active operating mode.

Here is the caller graph for this function:

4.1.3.2.6 ElevatorController_u8GetCurrentFloor()

```
uint8_t ElevatorController_u8GetCurrentFloor (
    void )
```

Retrieves the current floor position of the elevator.

Returns

[uint8_t](#) Zero-based floor index where the cabin is located.

Retrieves the current floor position of the elevator. Here is the caller graph for this function:

4.1.3.2.7 ElevatorController_u8GetSelectorCnt()

```
uint8_t ElevatorController_u8GetSelectorCnt (
    void )
```

Retrieves the selector pole count accumulated since the last reset.

The selector pole count is used for floor position tracking via the selector sensor.

Returns

uint8_t Current selector pole counter value.

Retrieves the selector pole count accumulated since the last reset.

Returns

uint8_t

Here is the caller graph for this function:

4.1.3.2.8 ElevatorController_vidOperationLoop()

```
void ElevatorController_vidOperationLoop (
    void )
```

Main periodic operation loop of the elevator controller.

Must be called repeatedly in normal service mode. Handles call collection, motion control, door management, and LED updates. Here is the call graph for this function: Here is the caller graph for this function:

4.1.3.2.9 ElevatorController_vidProgrammingLoop()

```
void ElevatorController_vidProgrammingLoop (
    void )
```

Programming loop of the elevator controller.

Executed when the system boots into programming mode. Allows the technician to configure persistent parameters (timers, floor count, door options, etc.) via the on-board menu. Here is the call graph for this function: Here is the caller graph for this function:

4.1.3.2.10 ElevatorController_vidStartLightTimer()

```
void ElevatorController_vidStartLightTimer (
    void )
```

Starts (or restarts) the cabin light auto-off timer.

Call this whenever a cabin door or call button event occurs to keep the cabin light on for the configured duration.

Starts (or restarts) the cabin light auto-off timer. Here is the caller graph for this function:

4.1.3.3 ElevatorController Local Config

Collaboration diagram for ElevatorController Local Config:

Macros

- `#define DOOR_OPEN_TIME_MS 3000`
Duration the door remains open before auto-close is triggered (ms).

Enumerations

- enum `CollectionDir_t` { `UP_COLLECTIVE` = 0 , `DOWN_COLLECTIVE` }
Direction used for the collective call algorithm.

EEPROM Default Values

Factory-default values written to EEPROM on first power-up.

- `#define cu8SLOW_TIMER_DEF_VALUE ((uint8_t) 7U)`
- `#define cu8FAST_TIMER_DEF_VALUE ((uint8_t) 18U)`
- `#define cu8CAM_TIMER_DEF_VALUE ((uint8_t) 3U)`
- `#define cu8STOP_TIMER_DEF_VALUE ((uint8_t) 2U)`
- `#define cu8LIGHT_TIMER_DEF_VALUE ((uint8_t) 10U)`
- `#define cu8DOOR_NUMBER_DEF_VALUE ((uint8_t) 16U)`
- `#define cu8DISPLAY_DEF_VALUE ((uint8_t) 0U)`
- `#define cu8FLOOR_0_DEF_VALUE ((uint8_t) 0U)`
- `#define cu8FLOOR_1_DEF_VALUE ((uint8_t) 1U)`
- `#define cu8FLOOR_2_DEF_VALUE ((uint8_t) 2U)`
- `#define cu8FLOOR_3_DEF_VALUE ((uint8_t) 3U)`
- `#define cu8FLOOR_4_DEF_VALUE ((uint8_t) 4U)`
- `#define cu8FLOOR_5_DEF_VALUE ((uint8_t) 5U)`
- `#define cu8FLOOR_6_DEF_VALUE ((uint8_t) 6U)`
- `#define cu8FLOOR_7_DEF_VALUE ((uint8_t) 7U)`
- `#define cu8FLOOR_8_DEF_VALUE ((uint8_t) 8U)`
- `#define cu8FLOOR_9_DEF_VALUE ((uint8_t) 9U)`
- `#define cu8FLOOR_10_DEF_VALUE ((uint8_t) 10U)`
- `#define cu8FLOOR_11_DEF_VALUE ((uint8_t) 11U)`
- `#define cu8FLOOR_12_DEF_VALUE ((uint8_t) 12U)`
- `#define cu8FLOOR_13_DEF_VALUE ((uint8_t) 13U)`
- `#define cu8FLOOR_14_DEF_VALUE ((uint8_t) 14U)`
- `#define cu8FLOOR_15_DEF_VALUE ((uint8_t) 15U)`
- `#define cu8COLLECTION_DIR_DEF_VALUE ((uint8_t) DOWN_COLLECTIVE)`
- `#define cu8MNT_SPEED_DEF_VALUE ((uint8_t) 0U)`
- `#define cu8DOOR_OPTIONS_DEF_VALUE ((uint8_t) 0U)`
- `#define cu8PARKING_FLOOR_DEF_VALUE ((uint8_t) 19U)`
- `#define cu8CAM_FAIL_CNT_DEF_VALUE ((uint8_t) 9U)`
- `#define cu8CABIN_PW_DEF_VALUE ((uint8_t) 0U)`
- `#define cu8PHS_SEQ_DEF_VALUE ((uint8_t) 0U)`
- `#define cu8PASSWORD_L_DEF_VALUE ((uint8_t) 0U)`
- `#define cu8PASSWORD_H_DEF_VALUE ((uint8_t) 0U)`
- `#define cu8CURRENT_FLOOR_DEF_VALUE ((uint8_t) 0U)`
- `#define cu8CURRENT_TRAVEL_L_DEF_VALUE ((uint8_t) 0U)`
- `#define cu8CURRENT_TRAVEL_H_DEF_VALUE ((uint8_t) 0U)`
- `#define cu8STORED_TRAVEL_DEF_VALUE ((uint8_t) 10U)`
- `#define cu8STORED_TRAVEL_USAGE_DEF_VALUE ((uint8_t) 0U)`
- `#define cu8MASTER_PASSWORD_L_DEF_VALUE ((uint8_t) 158U)`
- `#define cu8MASTER_PASSWORD_H_DEF_VALUE ((uint8_t) 0U)`

EEPROM Minimum Values

Lower bounds enforced when saving parameters via the programming menu.

- `#define cu8SLOW_TIMER_MIN_VALUE ((uint8_t) 1U)`
- `#define cu8FAST_TIMER_MIN_VALUE ((uint8_t) 1U)`
- `#define cu8CAM_TIMER_MIN_VALUE ((uint8_t) 1U)`
- `#define cu8STOP_TIMER_MIN_VALUE ((uint8_t) 1U)`
- `#define cu8LIGHT_TIMER_MIN_VALUE ((uint8_t) 1U)`
- `#define cu8DOOR_NUMBER_MIN_VALUE ((uint8_t) 1U)`
- `#define cu8COLLECTION_DIR_MIN_VALUE ((uint8_t) DOWN_COLLECTIVE)`
- `#define cu8MNT_SPEED_MIN_VALUE ((uint8_t) 0U)`
- `#define cu8DOOR_OPTIONS_MIN_VALUE ((uint8_t) 0U)`
- `#define cu8PARKING_FLOOR_MIN_VALUE ((uint8_t) 0U)`
- `#define cu8CAM_FAIL_CNT_MIN_VALUE ((uint8_t) 0U)`
- `#define cu8CABIN_PW_MIN_VALUE ((uint8_t) 0U)`
- `#define cu8PHS_SEQ_MIN_VALUE ((uint8_t) 0U)`

EEPROM Maximum Values

Upper bounds enforced when saving parameters via the programming menu.

- `#define cu8SLOW_TIMER_MAX_VALUE ((uint8_t) 20U)`
- `#define cu8FAST_TIMER_MAX_VALUE ((uint8_t) 30U)`
- `#define cu8CAM_TIMER_MAX_VALUE ((uint8_t) 20U)`
- `#define cu8STOP_TIMER_MAX_VALUE ((uint8_t) 10U)`
- `#define cu8LIGHT_TIMER_MAX_VALUE ((uint8_t) 20U)`
- `#define cu8DOOR_NUMBER_MAX_VALUE ((uint8_t) 16U)`
- `#define cu8COLLECTION_DIR_MAX_VALUE ((uint8_t) 4U)`
- `#define cu8MNT_SPEED_MAX_VALUE ((uint8_t) 1U)`
- `#define cu8DOOR_OPTIONS_MAX_VALUE ((uint8_t) 1U)`
- `#define cu8PARKING_FLOOR_MAX_VALUE (cu8DOOR_NUMBER_MAX_VALUE)`
- `#define cu8CAM_FAIL_CNT_MAX_VALUE ((uint8_t) 20U)`
- `#define cu8CABIN_PW_MAX_VALUE ((uint8_t) 1U)`
- `#define cu8PHS_SEQ_MAX_VALUE ((uint8_t) 1U)`

4.1.3.3.1 Detailed Description

4.1.3.3.2 Macro Definition Documentation

4.1.3.3.2.1 cu8CABIN_PW_DEF_VALUE

```
#define cu8CABIN_PW_DEF_VALUE ((uint8_t) 0U)
```

Default cabin password.

4.1.3.3.2.2 cu8CAM_FAIL_CNT_DEF_VALUE

```
#define cu8CAM_FAIL_CNT_DEF_VALUE ((uint8_t) 9U)
```

Default CAM failure count threshold.

4.1.3.3.2.3 cu8CAM_TIMER_DEF_VALUE

```
#define cu8CAM_TIMER_DEF_VALUE ((uint8_t) 3U)
```

Default CAM pulse timer.

4.1.3.3.2.4 cu8COLLECTION_DIR_DEF_VALUE

```
#define cu8COLLECTION_DIR_DEF_VALUE ((uint8_t) DOWN_COLLECTIVE)
```

Default collection direction.

4.1.3.3.2.5 cu8CURRENT_FLOOR_DEF_VALUE

```
#define cu8CURRENT_FLOOR_DEF_VALUE ((uint8_t) 0U)
```

Default stored current floor.

4.1.3.3.2.6 cu8CURRENT_TRAVEL_H_DEF_VALUE

```
#define cu8CURRENT_TRAVEL_H_DEF_VALUE ((uint8_t) 0U)
```

Default travel counter high byte.

4.1.3.3.2.7 cu8CURRENT_TRAVEL_L_DEF_VALUE

```
#define cu8CURRENT_TRAVEL_L_DEF_VALUE ((uint8_t) 0U)
```

Default travel counter low byte.

4.1.3.3.2.8 cu8DISPLAY_DEF_VALUE

```
#define cu8DISPLAY_DEF_VALUE ((uint8_t) 0U)
```

Default display setting.

4.1.3.3.2.9 cu8DOOR_NUMBER_DEF_VALUE

```
#define cu8DOOR_NUMBER_DEF_VALUE ((uint8_t) 16U)
```

Default number of doors / floors.

4.1.3.3.2.10 cu8DOOR_OPTIONS_DEF_VALUE

```
#define cu8DOOR_OPTIONS_DEF_VALUE ((uint8_t) 0U)
```

Default door options byte.

4.1.3.3.2.11 cu8FAST_TIMER_DEF_VALUE

```
#define cu8FAST_TIMER_DEF_VALUE ((uint8_t) 18U)
```

Default fast (high-speed) timer.

4.1.3.3.2.12 cu8FLOOR_0_DEF_VALUE

```
#define cu8FLOOR_0_DEF_VALUE ((uint8_t) 0U)
```

Default floor 0 label.

4.1.3.3.2.13 cu8FLOOR_10_DEF_VALUE

```
#define cu8FLOOR_10_DEF_VALUE ((uint8_t) 10U)
```

Default floor 10 label.

4.1.3.3.2.14 cu8FLOOR_11_DEF_VALUE

```
#define cu8FLOOR_11_DEF_VALUE ((uint8_t) 11U)
```

Default floor 11 label.

4.1.3.3.2.15 cu8FLOOR_12_DEF_VALUE

```
#define cu8FLOOR_12_DEF_VALUE ((uint8_t) 12U)
```

Default floor 12 label.

4.1.3.3.2.16 cu8FLOOR_13_DEF_VALUE

```
#define cu8FLOOR_13_DEF_VALUE ((uint8_t) 13U)
```

Default floor 13 label.

4.1.3.3.2.17 cu8FLOOR_14_DEF_VALUE

```
#define cu8FLOOR_14_DEF_VALUE ((uint8_t) 14U)
```

Default floor 14 label.

4.1.3.3.2.18 cu8FLOOR_15_DEF_VALUE

```
#define cu8FLOOR_15_DEF_VALUE ((uint8_t) 15U)
```

Default floor 15 label.

4.1.3.3.2.19 cu8FLOOR_1_DEF_VALUE

```
#define cu8FLOOR_1_DEF_VALUE ((uint8_t) 1U)
```

Default floor 1 label.

4.1.3.3.2.20 cu8FLOOR_2_DEF_VALUE

```
#define cu8FLOOR_2_DEF_VALUE ((uint8_t) 2U)
```

Default floor 2 label.

4.1.3.3.2.21 cu8FLOOR_3_DEF_VALUE

```
#define cu8FLOOR_3_DEF_VALUE ((uint8_t) 3U)
```

Default floor 3 label.

4.1.3.3.2.22 cu8FLOOR_4_DEF_VALUE

```
#define cu8FLOOR_4_DEF_VALUE ((uint8_t) 4U)
```

Default floor 4 label.

4.1.3.3.2.23 cu8FLOOR_5_DEF_VALUE

```
#define cu8FLOOR_5_DEF_VALUE ((uint8_t) 5U)
```

Default floor 5 label.

4.1.3.3.2.24 cu8FLOOR_6_DEF_VALUE

```
#define cu8FLOOR_6_DEF_VALUE ((uint8_t) 6U)
```

Default floor 6 label.

4.1.3.3.2.25 cu8FLOOR_7_DEF_VALUE

```
#define cu8FLOOR_7_DEF_VALUE ((uint8_t) 7U)
```

Default floor 7 label.

4.1.3.3.2.26 cu8FLOOR_8_DEF_VALUE

```
#define cu8FLOOR_8_DEF_VALUE ((uint8_t) 8U)
```

Default floor 8 label.

4.1.3.3.2.27 cu8FLOOR_9_DEF_VALUE

```
#define cu8FLOOR_9_DEF_VALUE ((uint8_t) 9U)
```

Default floor 9 label.

4.1.3.3.2.28 cu8LIGHT_TIMER_DEF_VALUE

```
#define cu8LIGHT_TIMER_DEF_VALUE ((uint8_t) 10U)
```

Default cabin light auto-off timer.

4.1.3.3.2.29 cu8MASTER_PASSWORD_H_DEF_VALUE

```
#define cu8MASTER_PASSWORD_H_DEF_VALUE ((uint8_t) 0U)
```

Master password high byte.

4.1.3.3.2.30 cu8MASTER_PASSWORD_L_DEF_VALUE

```
#define cu8MASTER_PASSWORD_L_DEF_VALUE ((uint8_t) 158U)
```

Master password low byte.

4.1.3.3.2.31 cu8MNT_SPEED_DEF_VALUE

```
#define cu8MNT_SPEED_DEF_VALUE ((uint8_t) 0U)
```

Default maintenance speed index.

4.1.3.3.2.32 cu8PARKING_FLOOR_DEF_VALUE

```
#define cu8PARKING_FLOOR_DEF_VALUE ((uint8_t) 19U)
```

Default parking floor (disabled = 19).

4.1.3.3.2.33 cu8PASSWORD_H_DEF_VALUE

```
#define cu8PASSWORD_H_DEF_VALUE ((uint8_t) 0U)
```

Default access password high byte.

4.1.3.3.2.34 cu8PASSWORD_L_DEF_VALUE

```
#define cu8PASSWORD_L_DEF_VALUE ((uint8_t) 0U)
```

Default access password low byte.

4.1.3.3.2.35 cu8PHS_SEQ_DEF_VALUE

```
#define cu8PHS_SEQ_DEF_VALUE ((uint8_t) 0U)
```

Default phase sequence byte.

4.1.3.3.2.36 cu8SLOW_TIMER_DEF_VALUE

```
#define cu8SLOW_TIMER_DEF_VALUE ((uint8_t) 7U)
```

Default slow (decel) timer.

4.1.3.3.2.37 cu8STOP_TIMER_DEF_VALUE

```
#define cu8STOP_TIMER_DEF_VALUE ((uint8_t) 2U)
```

Default stop phase timer.

4.1.3.3.2.38 cu8STORED_TRAVEL_DEF_VALUE

```
#define cu8STORED_TRAVEL_DEF_VALUE ((uint8_t) 10U)
```

Default stored travel distance.

4.1.3.3.2.39 cu8STORED_TRAVEL_USAGE_DEF_VALUE

```
#define cu8STORED_TRAVEL_USAGE_DEF_VALUE ((uint8_t) 0U)
```

Default travel usage counter.

4.1.3.3.3 Enumeration Type Documentation**4.1.3.3.3.1 CollectionDir_t**

```
enum CollectionDir_t
```

Direction used for the collective call algorithm.

Enumerator

UP_COLLECTIVE	Service calls in the upward direction first.
DOWN_COLLECTIVE	Service calls in the downward direction first.

4.1.3.4 ElevatorController Global Config

Collaboration diagram for ElevatorController Global Config:

4.1.4 LEDController

Collaboration diagram for LEDController:

Topics

- [LEDController Global Config](#)

Functions

- void [LEDController_vidInit](#) (void)
Initializes the LEDController module.
- void [LEDController_vidSetState](#) (uint8_t u8LedId, [LEDState_t](#) enuState)
Sets the instantaneous state of a specific LED.
- void [LEDController_vidTurnAllOff](#) (void)
Turns all LEDs off immediately.
- void [LEDController_vidSetPatAllOff](#) (void)
Clears the blink pattern for all LEDs.
- void [LEDController_vidSetPattern](#) (uint8_t u8LedId, [LEDPattern_t](#) enuPattern)
Assigns a blink pattern to a specific LED.
- void [LEDController_vidProcess](#) (void)
Periodic processing task for the LED controller.

4.1.4.1 Detailed Description

4.1.4.2 Function Documentation

4.1.4.2.1 LEDController_vidInit()

```
void LEDController_vidInit (
    void )
```

Initializes the LEDController module.

Configures the latch and data GPIO lines as outputs, clears all latch outputs, and resets internal LED state tracking.

Initializes the LEDController module. Here is the caller graph for this function:

4.1.4.2.2 LEDController_vidProcess()

```
void LEDController_vidProcess (
    void )
```

Periodic processing task for the LED controller.

Updates blink timers and writes the resulting LED states to the hardware latches. Must be called every [cu8LED_CONTROLLER_PERIOD_MS](#) milliseconds.

Periodic processing task for the LED controller. Here is the caller graph for this function:

4.1.4.2.3 LEDController_vidSetPatAllOff()

```
void LEDController_vidSetPatAllOff (
    void )
```

Clears the blink pattern for all LEDs.

Sets every LED's pattern to [LED_PATTERN_NONE](#) without changing the current physical on/off state.

Clears the blink pattern for all LEDs. Here is the caller graph for this function:

4.1.4.2.4 LEDController_vidSetPattern()

```
void LEDController_vidSetPattern (
    uint8_t u8LedId,
    LEDPattern_t enuPattern)
```

Assigns a blink pattern to a specific LED.

The pattern determines the on/off timing applied when the LED is processed by [LEDController_vidProcess](#).

Parameters

<i>u8LedId</i>	Logical LED index (0 to cu8MAX_FLOORS - 1).
<i>enuPattern</i>	Blink pattern to assign (see LEDPattern_t).

Assigns a blink pattern to a specific LED.

Parameters

<i>u8LedId</i>	
<i>enuPattern</i>	

Here is the caller graph for this function:

4.1.4.2.5 LEDController_vidSetState()

```
void LEDController_vidSetState (
    uint8_t u8LedId,
    LEDState_t enuState)
```

Sets the instantaneous state of a specific LED.

Parameters

<i>u8LedId</i>	Logical LED index (0 to cu8MAX_FLOORS - 1).
----------------	---

<i>enuState</i>	Desired state: LED_STATE_OFF , LED_STATE_ON , or LED_STATE_BLINKING .
-----------------	---

Sets the instantaneous state of a specific LED.

Parameters

<i>u8LedId</i>	
<i>enuState</i>	

Here is the caller graph for this function:

4.1.4.2.6 LEDController_vidTurnAllOff()

```
void LEDController_vidTurnAllOff (
    void )
```

Turns all LEDs off immediately.

Writes a cleared state to both latches and resets all internal active flags.

Turns all LEDs off immediately. Here is the caller graph for this function:

4.1.4.3 LEDController Global Config

Collaboration diagram for LEDController Global Config:

Data Structures

- struct [LEDHWConfig_t](#)
Maps a logical LED index to its physical latch and bit position.
- struct [LEDConfig_t](#)
Configuration entry for a single LED pattern.
- struct [FloorLED_t](#)
Runtime state of a single floor LED.

Macros

- `#define cu8LED_CONTROLLER_PERIOD_MS ((uint8_t) 10U)`
Task period for the LED controller (ms).

Enumerations

- enum `LEDCallType_t` { `LED_CALL_TYPE_INTERNAL` , `LED_CALL_TYPE_EXTERNAL` }
Classifies the call type associated with an LED.
- enum `LEDState_t` { `LED_STATE_OFF` , `LED_STATE_ON` , `LED_STATE_BLINKING` }
Represents the instantaneous state of a single LED.
- enum `LEDPattern_t` {
 `LED_PATTERN_NONE` , `LED_PATTERN_INTERNAL_CALL` , `LED_PATTERN_EXTERNAL_CALL` ,
 `LED_PATTERN_MAINTENANCE` ,
 `LED_PATTERN_ERROR` }
Pre-defined blink patterns for floor indicator LEDs.

Variables

- const `LEDHWConfig_t LED_HWConfig` [cu8MAX_FLOORS]
External declaration of the per-floor hardware configuration array.

Latch Control Pins

GPIO port and pin assignments for the two 74HC573 latch enable lines.

- #define `cenuLED_LATCH0_PORT` ((GPIO_t) GPIOB)
- #define `cu8LED_LATCH0_PIN` ((uint8_t) 2U)
- #define `cenuLED_LATCH1_PORT` ((GPIO_t) GPIOB)
- #define `cu8LED_LATCH1_PIN` ((uint8_t) 3U)

LED Data Port

8-bit parallel data port used to write LED states to the active latch.

- #define `cenuLED_DATA_PORT` ((GPIO_t) GPIOA)

LED Pattern Timing

On-time and off-time constants (in milliseconds) for each LED pattern.

- #define `cu16LED_INTERNAL_CALL_ON_TIME` ((uint16_t) 60000U)
- #define `cu16LED_INTERNAL_CALL_OFF_TIME` ((uint16_t) 0U)
- #define `cu16LED_EXTERNAL_CALL_ON_TIME` ((uint16_t) 5000U)
- #define `cu16LED_EXTERNAL_CALL_OFF_TIME` ((uint16_t) 5000U)
- #define `cu16LED_MAINTENANCE_ON_TIME` ((uint16_t) 200U)
- #define `cu16LED_MAINTENANCE_OFF_TIME` ((uint16_t) 200U)
- #define `cu16LED_ERROR_ON_TIME` ((uint16_t) 100U)
- #define `cu16LED_ERROR_OFF_TIME` ((uint16_t) 100U)

4.1.4.3.1 Detailed Description

4.1.4.3.2 Macro Definition Documentation

4.1.4.3.2.1 `cenuLED_DATA_PORT`

```
#define cenuLED_DATA_PORT ((GPIO_t) GPIOA)
```

Port carrying the 8-bit LED data bus.

4.1.4.3.2.2 `cenuLED_LATCH0_PORT`

```
#define cenuLED_LATCH0_PORT ((GPIO_t) GPIOB)
```

Port for latch 0 enable (LEDs 0-7).

4.1.4.3.2.3 `cenuLED_LATCH1_PORT`

```
#define cenuLED_LATCH1_PORT ((GPIO_t) GPIOB)
```

Port for latch 1 enable (LEDs 8-15).

4.1.4.3.2.4 `cu16LED_ERROR_OFF_TIME`

```
#define cu16LED_ERROR_OFF_TIME ((uint16_t) 100U)
```

Error mode LED off time (ms).

4.1.4.3.2.5 `cu16LED_ERROR_ON_TIME`

```
#define cu16LED_ERROR_ON_TIME ((uint16_t) 100U)
```

Error mode LED on time (ms).

4.1.4.3.2.6 `cu16LED_EXTERNAL_CALL_OFF_TIME`

```
#define cu16LED_EXTERNAL_CALL_OFF_TIME ((uint16_t) 5000U)
```

External call LED off time (ms).

4.1.4.3.2.7 `cu16LED_EXTERNAL_CALL_ON_TIME`

```
#define cu16LED_EXTERNAL_CALL_ON_TIME ((uint16_t) 5000U)
```

External call LED on time (ms).

4.1.4.3.2.8 cu16LED_INTERNAL_CALL_OFF_TIME

```
#define cu16LED_INTERNAL_CALL_OFF_TIME ((uint16_t) 0U)
```

Internal call LED off time (ms).

4.1.4.3.2.9 cu16LED_INTERNAL_CALL_ON_TIME

```
#define cu16LED_INTERNAL_CALL_ON_TIME ((uint16_t) 60000U)
```

Internal call LED on time (ms).

4.1.4.3.2.10 cu16LED_MAINTENANCE_OFF_TIME

```
#define cu16LED_MAINTENANCE_OFF_TIME ((uint16_t) 200U)
```

Maintenance mode LED off time (ms).

4.1.4.3.2.11 cu16LED_MAINTENANCE_ON_TIME

```
#define cu16LED_MAINTENANCE_ON_TIME ((uint16_t) 200U)
```

Maintenance mode LED on time (ms).

4.1.4.3.2.12 cu8LED_LATCH0_PIN

```
#define cu8LED_LATCH0_PIN ((uint8_t) 2U)
```

Pin number for latch 0 enable.

4.1.4.3.2.13 cu8LED_LATCH1_PIN

```
#define cu8LED_LATCH1_PIN ((uint8_t) 3U)
```

Pin number for latch 1 enable.

4.1.4.3.3 Enumeration Type Documentation

4.1.4.3.3.1 LEDCallType_t

enum [LEDCallType_t](#)

Classifies the call type associated with an LED.

Enumerator

LED_CALL_TYPE_INTERNAL	LED corresponds to an internal (cabin) call.
LED_CALL_TYPE_EXTERNAL	LED corresponds to an external (landing) call.

4.1.4.3.3.2 LEDPattern_t

enum [LEDPattern_t](#)

Pre-defined blink patterns for floor indicator LEDs.

Enumerator

LED_PATTERN_NONE	No active pattern; LED follows its direct state.
LED_PATTERN_INTERNAL_CALL	Pattern for a registered internal call.
LED_PATTERN_EXTERNAL_CALL	Pattern for a registered external call.
LED_PATTERN_MAINTENANCE	Pattern used during maintenance mode.
LED_PATTERN_ERROR	Pattern used during error mode.

4.1.4.3.3.3 LEDState_t

enum [LEDState_t](#)

Represents the instantaneous state of a single LED.

Enumerator

LED_STATE_OFF	LED is off.
LED_STATE_ON	LED is continuously on.
LED_STATE_BLINKING	LED is blinking according to its assigned pattern.

4.1.5 Main

Collaboration diagram for Main:

Macros

- #define **F_CPU** 8000000UL

Functions

- int [main](#) (void)
Application entry point.

4.1.5.1 Detailed Description

4.1.5.2 Function Documentation

4.1.5.2.1 main()

```
int main (
    void )
```

Application entry point.

Sequence:

1. Initialize all hardware and software sub-modules via [ElevatorController_Init](#).
2. Display the splash screen via [ElevatorController_vidSplashScreen](#).
3. Read the boot mode via [ElevatorController_enuGetMode](#).
4. If MODE_PROGRAM: enter [ElevatorController_vidProgrammingLoop](#).
5. If MODE_NORMAL: enter [ElevatorController_vidOperationLoop](#).

Returns

int Always returns 0 (never reached on embedded target).

Here is the call graph for this function:

4.1.6 Menu

Collaboration diagram for Menu:

Topics

- [Menu Global Config](#)

Functions

- void [Menu_Init](#) (void)
Initializes the menu system.
- void [Menu_Task](#) (void)
Periodic task handler for the menu system.
- void [Menu_Update](#) (void)
Processes user input and updates the menu state.
- void [Menu_SetItemData](#) ([MenuItemId_t](#) itemId, const char *label, uint8_t value, uint8_t minValue, uint8_t maxValue, uint8_t eepromAddress)
Configures a menu item with its label, default value, and bounds.
- void [Menu_UpdateItemValue](#) ([MenuItemId_t](#) itemId, uint8_t value)
Updates the stored value of an existing menu item.

4.1.6.1 Detailed Description

4.1.6.2 Function Documentation

4.1.6.2.1 Menu_Init()

```
void Menu_Init (
    void )
```

Initializes the menu system.

Must be called once before any other Menu function. Resets the item list, sets the navigation cursor to the first item, and sets the mode to [MENU_MODE_NAVIGATE](#). Here is the caller graph for this function:

4.1.6.2.2 Menu_SetItemData()

```
void Menu_SetItemData (
    MenuItemId_t itemId,
    const char * label,
    uint8_t value,
    uint8_t minValue,
    uint8_t maxValue,
    uint8_t eepromAddress)
```

Configures a menu item with its label, default value, and bounds.

Parameters

<i>itemId</i>	Unique identifier of the menu item (MenuItemId_t).
<i>label</i>	Null-terminated display label for the item.
<i>value</i>	Initial (current) value loaded from EEPROM.
<i>minValue</i>	Minimum allowable value for this parameter.
<i>maxValue</i>	Maximum allowable value for this parameter.
<i>eepromAddress</i>	EEPROM address where this parameter is stored.

4.1.6.2.3 Menu_Task()

```
void Menu_Task (
    void )
```

Periodic task handler for the menu system.

Handles timed events such as cursor blinking. Should be called every [MENU_BLINK_PERIOD_MS](#) milliseconds. Here is the caller graph for this function:

4.1.6.2.4 Menu_Update()

```
void Menu_Update (
    void )
```

Processes user input and updates the menu state.

Reads button states to navigate between menu items or increment / decrement the selected item's value while in edit mode. Writes confirmed values to EEPROM. Here is the caller graph for this function:

4.1.6.2.5 Menu_UpdateItemValue()

```
void Menu_UpdateItemValue (
    MenuItemId_t itemId,
    uint8_t value)
```

Updates the stored value of an existing menu item.

Refreshes the runtime value without changing the label or bounds. Useful for reloading a parameter from EEPROM after an external write.

Parameters

<i>itemId</i>	Unique identifier of the menu item (MenuItemId_t).
<i>value</i>	New value to assign to the item.

4.1.6.3 Menu Global Config

Collaboration diagram for Menu Global Config:

Data Structures

- struct [MenuItem_t](#)
Data record for a single configurable menu item.

Macros

- #define **MENU_MAX_ITEMS** ((uint8_t) 10U)
Maximum number of menu items allowed in the system.
- #define **MENU_BLINK_PERIOD_MS** ((uint16_t) 20U)
Blink period for the selected menu item cursor (ms).

Enumerations

- enum [MenuItemId_t](#) {
[SLOW_TIMER_MID](#) , [FAST_TIMER_MID](#) , [CAM_TIMER_MID](#) , [STOP_TIMER_MID](#) ,
[LIGHT_TIMER_MID](#) , [DOOR_NUMBER_MID](#) , [COLLECTION_DIR_MID](#) , [MNT_SPEED_MID](#) ,
[DOOR_OPTIONS_MID](#) , [PARKING_FLOOR_MID](#) , [CAM_FAIL_CNT_MID](#) , [CABIN_PW_MID](#) ,
[PHS_SEQ_MID](#) , [MENU_ITEM_COUNT](#) }
Unique identifiers for each configurable menu item.
- enum [MenuMode_t](#) { [MENU_MODE_NAVIGATE](#) , [MENU_MODE_EDIT](#) }
Operating mode of the menu navigation state machine.

4.1.6.3.1 Detailed Description

4.1.6.3.2 Enumeration Type Documentation

4.1.6.3.2.1 MenuItemId_t

enum [MenuItemId_t](#)

Unique identifiers for each configurable menu item.

Enumerator

SLOW_TIMER_MID	Deceleration timer configuration item.
FAST_TIMER_MID	High-speed travel timer configuration item.
CAM_TIMER_MID	CAM pulse timer configuration item.
STOP_TIMER_MID	Stop phase timer configuration item.
LIGHT_TIMER_MID	Cabin light auto-off timer configuration item.
DOOR_NUMBER_MID	Door count / door number configuration item.
COLLECTION_DIR_MID	Call collection direction configuration item.
MNT_SPEED_MID	Maintenance speed configuration item.
DOOR_OPTIONS_MID	Door type options configuration item.
PARKING_FLOOR_MID	Parking floor configuration item.
CAM_FAIL_CNT_MID	CAM failure count threshold configuration item.
CABIN_PW_MID	Cabin panel access password configuration item.
PHS_SEQ_MID	Phase sequence configuration item.
MENU_ITEM_COUNT	Total number of menu items (not a valid item ID).

4.1.6.3.2.2 MenuMode_t

enum [MenuMode_t](#)

Operating mode of the menu navigation state machine.

Enumerator

MENU_MODE_NAVIGATE	User is scrolling through menu items.
MENU_MODE_EDIT	User is editing the value of the selected item.

4.1.7 MotionController

Collaboration diagram for MotionController:

Topics

- [MotionController Global Config](#)

Functions

- void [MotionController_vidInit](#) (void)
Initializes the MotionController module.
- void [MotionController_vidMove](#) ([MotionState_t](#) enuState)
Applies a motion command to the elevator drive.

4.1.7.1 Detailed Description

4.1.7.2 Function Documentation

4.1.7.2.1 MotionController_vidInit()

```
void MotionController_vidInit (
    void )
```

Initializes the MotionController module.

Configures the relay output lines and sets the drive to the [MOTION_STATE_IDLE](#) state.

Initializes the MotionController module. Here is the caller graph for this function:

4.1.7.2.2 MotionController_vidMove()

```
void MotionController_vidMove (
    MotionState\_t enuState)
```

Applies a motion command to the elevator drive.

Activates the relay combination that corresponds to the requested [MotionState_t](#). Only one motion state is active at a time; calling this function while the drive is running will switch states immediately.

Parameters

<i>enuState</i>	Target motion state (see MotionState_t).
-----------------	---

Applies a motion command to the elevator drive.

Parameters

<i>enuState</i>	The motion state and speed (up, down or stop).
-----------------	--

4.1.7.3 MotionController Global Config

Collaboration diagram for MotionController Global Config:

Enumerations

- enum `MotionState_t` {
`MOTION_STATE_IDLE` , `MOTION_STATE_MOVING_UP_HS` , `MOTION_STATE_MOVING_UP_LS` ,
`MOTION_STATE_MOVING_DOWN_HS` ,
`MOTION_STATE_MOVING_DOWN_LS` , `MOTION_STATE_STOPPING` , `MOTION_STATE_ERROR` }

Defines the motion states for the elevator drive system.

4.1.7.3.1 Detailed Description

4.1.7.3.2 Enumeration Type Documentation

4.1.7.3.2.1 MotionState_t

enum `MotionState_t`

Defines the motion states for the elevator drive system.

Enumerator

<code>MOTION_STATE_IDLE</code>	Drive is stopped with no pending command.
<code>MOTION_STATE_MOVING_UP_HS</code>	Moving upward at high speed.
<code>MOTION_STATE_MOVING_UP_LS</code>	Moving upward at low (slow) speed.
<code>MOTION_STATE_MOVING_DOWN_HS</code>	Moving downward at high speed.
<code>MOTION_STATE_MOVING_DOWN_LS</code>	Moving downward at low (slow) speed.
<code>MOTION_STATE_STOPPING</code>	Drive is executing a controlled stop sequence.
<code>MOTION_STATE_ERROR</code>	Drive is in a fault state.

Chapter 5

Data Structure Documentation

5.1 CallDioConfig_t Struct Reference

Data Fields

- GPIO_t enuCALL_0_Port
- uint8_t u8CALL_0_Pin
- GPIO_t enuCALL_1_Port
- uint8_t u8CALL_1_Pin
- GPIO_t enuSEL_0_Port
- uint8_t u8SEL_0_Pin
- GPIO_t enuSEL_1_Port
- uint8_t u8SEL_1_Pin
- GPIO_t enuSEL_2_Port
- uint8_t u8SEL_2_Pin
- GPIO_t enuCALL_CON_Port
- uint8_t u8CALL_CON_Pin

The documentation for this struct was generated from the following file:

- CallHandler/includes/CallHandler_lcfg.h

5.2 Elevator_t Struct Reference

Holds the full runtime state of the elevator.

```
#include <ElevatorController_gcfg.h>
```

Data Fields

- uint8_t [u8CurrentFloor](#)
- uint8_t [u8DestinationFloor](#)
- uint8_t [u8SlowTimer](#)
- uint8_t [u8FastTimer](#)
- uint8_t [u8CamTimer](#)
- uint8_t [u8StopTimer](#)
- uint8_t [u8LightTimer](#)
- uint8_t [u8FloorCount](#)
- uint8_t [u8MntSpeed](#)
- uint8_t [u8ParkFloor](#)
- uint8_t [u8CamFailCnt](#)
- uint8_t [u8CabinPwd](#)
- uint8_t [u8PhsSeq](#)
- uint8_t [u8CollectionDir](#)
- [MoveDirection_t](#) [enuDirection](#)
- [CollDir_t](#) [enuCollDir](#)
- [DoorType_t](#) [enuDoorType](#)
- [DoorState_t](#) [enuDoorState](#)
- [OperatingMode_t](#) [enuOperatingMode](#)
- [CallType_t](#) [aenuFloorCalls](#) [cu8MAX_FLOORS]
- uint16_t [u16DoorTimer](#)
- boolean [bEmergencyStop](#)

5.2.1 Detailed Description

Holds the full runtime state of the elevator.

5.2.2 Field Documentation

5.2.2.1 aenuFloorCalls

```
CallType\_t aenuFloorCalls[cu8MAX_FLOORS]
```

Array of pending call types indexed by floor.

5.2.2.2 bEmergencyStop

```
boolean bEmergencyStop
```

Flag indicating an active emergency stop condition.

5.2.2.3 enuCollDir

```
CollDir\_t enuCollDir
```

Active call-collection direction.

5.2.2.4 enuDirection

`MoveDirection_t` enuDirection

Current movement direction state.

5.2.2.5 enuDoorState

`DoorState_t` enuDoorState

Current door state.

5.2.2.6 enuDoorType

`DoorType_t` enuDoorType

Configured door type (manual or automatic).

5.2.2.7 enuOperatingMode

`OperatingMode_t` enuOperatingMode

Active operating mode of the controller.

5.2.2.8 u16DoorTimer

`uint16_t` u16DoorTimer

Door open timer countdown (in milliseconds).

5.2.2.9 u8CabinPwd

`uint8_t` u8CabinPwd

Cabin-panel access password.

5.2.2.10 u8CamFailCnt

`uint8_t` u8CamFailCnt

Count of consecutive CAM sensor failures.

5.2.2.11 u8CamTimer

`uint8_t` u8CamTimer

CAM pulse timer value.

5.2.2.12 u8CollectionDir

```
uint8_t u8CollectionDir
```

Raw collection direction byte.

5.2.2.13 u8CurrentFloor

```
uint8_t u8CurrentFloor
```

Current floor position of the elevator cabin.

5.2.2.14 u8DestinationFloor

```
uint8_t u8DestinationFloor
```

Target floor the elevator is moving toward.

5.2.2.15 u8FastTimer

```
uint8_t u8FastTimer
```

High-speed travel timer (configurable, in task ticks).

5.2.2.16 u8FloorCount

```
uint8_t u8FloorCount
```

Total number of floors in service.

5.2.2.17 u8LightTimer

```
uint8_t u8LightTimer
```

Cabin light auto-off timer (in task ticks).

5.2.2.18 u8MntSpeed

```
uint8_t u8MntSpeed
```

Maintenance mode travel speed setting.

5.2.2.19 u8ParkFloor

```
uint8_t u8ParkFloor
```

Parking floor where the elevator returns when idle.

5.2.2.20 u8PhsSeq

```
uint8_t u8PhsSeq
```

Phase sequence configuration byte.

5.2.2.21 u8SlowTimer

```
uint8_t u8SlowTimer
```

Deceleration timer (configurable, in task ticks).

5.2.2.22 u8StopTimer

```
uint8_t u8StopTimer
```

Timer used during the stop phase.

The documentation for this struct was generated from the following file:

- ElevatorController/interfaces/[ElevatorController_gcfg.h](#)

5.3 FloorLED_t Struct Reference

Runtime state of a single floor LED.

```
#include <LEDController_gcfg.h>
```

Data Fields

- boolean [blsActive](#)
- [LEDCallType_t](#) [enuLEDCallType](#)
- [uint16_t](#) [u16BlinkTimer](#)
- boolean [bLEDState](#)

5.3.1 Detailed Description

Runtime state of a single floor LED.

5.3.2 Field Documentation

5.3.2.1 blsActive

```
boolean bIsActive
```

TRUE if the LED has an active call or pattern.

5.3.2.2 bLEDState

`boolean bLEDState`

Current physical on/off output state.

5.3.2.3 enuLEDCallType

`LEDCallType_t enuLEDCallType`

Type of call driving this LED.

5.3.2.4 u16BlinkTimer

`uint16_t u16BlinkTimer`

Blink phase countdown timer (ms).

The documentation for this struct was generated from the following file:

- LEDController/interfaces/[LEDController_gcfg.h](#)

5.4 LEDConfig_t Struct Reference

Configuration entry for a single LED pattern.

```
#include <LEDController_gcfg.h>
```

Data Fields

- `uint8_t u8LedId`
- `LEDPattern_t enuPattern`
- `uint16_t u16OnTime`
- `uint16_t u16OffTime`

5.4.1 Detailed Description

Configuration entry for a single LED pattern.

5.4.2 Field Documentation

5.4.2.1 enuPattern

`LEDPattern_t enuPattern`

Active blink pattern assigned to this LED.

5.4.2.2 u16OffTime

```
uint16_t u16OffTime
```

Duration the LED stays off per cycle (ms).

5.4.2.3 u16OnTime

```
uint16_t u16OnTime
```

Duration the LED stays on per cycle (ms).

5.4.2.4 u8LedId

```
uint8_t u8LedId
```

Logical LED identifier.

The documentation for this struct was generated from the following file:

- LEDController/interfaces/[LEDController_gcfg.h](#)

5.5 LEDHWConfig_t Struct Reference

Maps a logical LED index to its physical latch and bit position.

```
#include <LEDController_gcfg.h>
```

Data Fields

- uint8_t [u8LatchIndex](#)
- uint8_t [u8BitPos](#)

5.5.1 Detailed Description

Maps a logical LED index to its physical latch and bit position.

5.5.2 Field Documentation

5.5.2.1 u8BitPos

```
uint8_t u8BitPos
```

Bit position within the latch (0-7).

5.5.2.2 u8LatchIndex

`uint8_t u8LatchIndex`

Latch index: 0 = first latch (LEDs 0-7), 1 = second latch (LEDs 8-15).

The documentation for this struct was generated from the following file:

- [LEDController/interfaces/LEDController_gcfg.h](#)

5.6 LEDStatus_t Struct Reference

Data Fields

- [LEDState_t](#) **enuState**
- [LEDPattern_t](#) **enuPattern**
- `uint16_t` **u16LastToggleTime**
- `boolean` **bCurrentOutput**
- `boolean` **blsInitialized**

The documentation for this struct was generated from the following file:

- [LEDController/includes/LEDController_lcfg.h](#)

5.7 MenuItem_t Struct Reference

Data record for a single configurable menu item.

```
#include <Menu_gcfg.h>
```

Data Fields

- `const char *` [label](#)
- `uint8_t` [value](#)
- `uint8_t` [minValue](#)
- `uint8_t` [maxValue](#)
- `uint8_t` [eepromAddress](#)

5.7.1 Detailed Description

Data record for a single configurable menu item.

5.7.2 Field Documentation

5.7.2.1 EEPROMAddress

```
uint8_t EEPROMAddress
```

EEPROM address where this parameter is persisted.

5.7.2.2 label

```
const char* label
```

Display label shown on the LCD for this item.

5.7.2.3 maxValue

```
uint8_t maxValue
```

Maximum allowable value.

5.7.2.4 minValue

```
uint8_t minValue
```

Minimum allowable value.

5.7.2.5 value

```
uint8_t value
```

Current value of the parameter.

The documentation for this struct was generated from the following file:

- Menu/interfaces/[Menu_gcfg.h](#)

Chapter 6

File Documentation

6.1 CallHandler_lcfcg.h

```
00001 /* ***** */
00002 /* ***** */
00003 /* (C) Copyright 2026 by */
00004 /* HexaMix */
00005 /* All rights reserved */
00006 /* This software is the exclusive property of HexaMix. */
00007 /* It cannot be communicated or divulged to anybody without a previous */
00008 /* written authorisation. */
00009 /* */
00010 /* Any partial or complete copy of this program whether it is in its */
00011 /* original shape or in its modified shape has to mention this copyright */
00012 /* and its proprietor. */
00013 /* */
00014 /* ***** */
00015 /* ***** */
00016
00017 /* ***** */
00018 /* ***** */
00019 /* FILE INCLUSION MANAGEMENT */
00020 /* ***** */
00021 /* ***** */
00022 /* Protection against bad and multiple inclusions */
00023 /* ***** */
00024
00025 #ifndef CALL_HANDLER_LCFG_H
00026 #define CALL_HANDLER_LCFG_H
00027
00028 /* ***** */
00029 /* Header Inclusions */
00030 /* ***** */
00031
00032 /* other modules of the project ***** */
00033 #include "dio_types.h"
00034 #include <stdint.h>
00035
00036 #if ! ((defined CallHandler_c) || (defined CallHandler_lcfcg_c))
00037 #error "This file must not be included from outside of it's component"
00038 #endif
00039
00040 /* ***** */
00041 /* ***** */
00042 /* CONFIGURATION DATA */
00043 /* ***** */
00044 /* ***** */
00045
00046 /* Call inputs GPIO Configuration */
00047 typedef struct {
00048     GPIO_t enuCALL_0_Port;
00049     uint8_t u8CALL_0_Pin;
00050     GPIO_t enuCALL_1_Port;
00051     uint8_t u8CALL_1_Pin;
00052     GPIO_t enuSEL_0_Port;
00053     uint8_t u8SEL_0_Pin;
00054     GPIO_t enuSEL_1_Port;
00055     uint8_t u8SEL_1_Pin;
00056     GPIO_t enuSEL_2_Port;
00057     uint8_t u8SEL_2_Pin;
```

```

00058     GPIO_t  enuCALL_CON_Port;
00059     uint8_t  u8CALL_CON_Pin;
00060 } CallDioConfig_t;
00061
00062 extern const CallDioConfig_t cstrCallDio_Config;
00063
00064 #endif // CALL_HANDLER_LCFG_H
00065
00066 /* ***** */
00067 /* ***** */
00068 /* ***** E N D   O F   F I L E ***** */
00069 /* ***** */
00070 /* ***** */

```

6.2 CallHandler/interfaces/CallHandler.h File Reference

Public interface for the CallHandler module.

```

#include "ElevatorController_gcfg.h"
#include "SystemConfig.h"
#include "CallHandler_gcfg.h"

```

Include dependency graph for CallHandler.h:

6.3 CallHandler.h

[Go to the documentation of this file.](#)

```

00001 /* ***** */
00002 /* ***** */
00003 /* (C) Copyright 2026 by */
00004 /* HexaMix */
00005 /* All rights reserved */
00006 /* This software is the exclusive property of HexaMix. */
00007 /* It cannot be communicated or divulged to anybody without a previous */
00008 /* written authorisation. */
00009 /* */
00010 /* Any partial or complete copy of this program whether it is in its */
00011 /* original shape or in its modified shape has to mention this copyright */
00012 /* and its proprietor. */
00013 /* */
00014 /* ***** */
00015 /* ***** */
00016
00017 /* ***** */
00018 /* ***** */
00019 /* FILE INCLUSION MANAGEMENT */
00020 /* ***** */
00021 /* ***** */
00022 /* Protection against bad and multiple inclusions */
00023 /* ***** */
00024
00025 #ifndef CALL_HANDLER_H
00026 #define CALL_HANDLER_H
00027
00040
00041 /* ***** */
00042 /* Header Inclusions */
00043 /* ***** */
00044
00045 /* cross dependency inclusions ***** */
00046 #include "ElevatorController_gcfg.h"
00047 #include "SystemConfig.h"
00048
00049 /* own header inclusions ***** */
00050
00051 /* export the global configuration */
00052 #include "CallHandler_gcfg.h"
00053
00054 /* ***** */
00055 /* ***** */
00056 /* Exported Functions */
00057 /* ***** */
00058 /* ***** */

```

```

00059
00060 /* Basic Services ***** */
00061
00062 /* Initialise the component. */
00071 void CallHandler_vidInit(Elevator_t* pstrElevator);
00072
00080 void CallHandler_vidGetCall(void);
00081
00083
00084 #endif // CALL_HANDLER_H
00085
00086 /* ***** */
00087 /* ***** */
00088 /* E N D O F F I L E */
00089 /* ***** */
00090 /* ***** */

```

6.4 CallHandler/interfaces/CallHandler_gcfg.h File Reference

Global configuration types for the CallHandler module.

```

#include "std_types.h"
#include "SystemConfig.h"

```

Include dependency graph for CallHandler_gcfg.h: This graph shows which files directly or indirectly include this file:

Enumerations

- enum [CallType_t](#) { [CALL_NONE](#) , [CALL_INTERNAL](#) , [CALL_EXTERNAL](#) }
- Classifies the type of a floor call.*

6.4.1 Detailed Description

Global configuration types for the CallHandler module.

Defines the [CallType_t](#) enumeration that classifies floor calls and is shared across application modules.

6.5 CallHandler_gcfg.h

[Go to the documentation of this file.](#)

```

00001 /* ***** */
00002 /* ***** */
00003 /* (C) Copyright 2026 by */
00004 /* HexaMix */
00005 /* All rights reserved */
00006 /* This software is the exclusive property of HexaMix. */
00007 /* It cannot be communicated or divulged to anybody without a previous */
00008 /* written authorisation. */
00009 /* */
00010 /* Any partial or complete copy of this program whether it is in its */
00011 /* original shape or in its modified shape has to mention this copyright */
00012 /* and its proprietor. */
00013 /* */
00014 /* ***** */
00015 /* ***** */
00016
00017 /* ***** */
00018 /* ***** */
00019 /* FILE INCLUSION MANAGEMENT */
00020 /* ***** */
00021 /* ***** */
00022 /* Protection against bad and multiple inclusions */

```

```

00023 /* ***** */
00024
00025 #ifndef CALL_HANDLER_GCFG_H
00026 #define CALL_HANDLER_GCFG_H
00027
00039
00040 #include "std_types.h"
00041 #include "SystemConfig.h"
00042
00043 /* ***** */
00044 /* ***** */
00045 /* Export of CallHandler configuration settings */
00046 /* ***** */
00047 /* ***** */
00048
00052 typedef enum
00053 {
00054     CALL_NONE,
00055     CALL_INTERNAL,
00056     CALL_EXTERNAL
00057 } CallType_t;
00058
00059 /*typedef struct
00060 {
00061     uint8_t u8Floor;           // Floor number (0 to MAX_FLOORS - 1)
00062     CallType_t enuCallType;    // Type of call (internal, external, or both)
00063     uint16_t u16Timestamp;     // Timestamp of the call (in milliseconds)
00064 } ElevatorCall_t;
00065
00066 typedef struct {
00067     ElevatorCall_t strCalls[cu8CALL_QUEUE_SIZE];
00068     uint8_t u8Head;
00069     uint8_t u8Tail;
00070     uint8_t u8Size;
00071 } CallQueue_t;
00072
00073 // LED state tracking
00074 typedef struct {
00075     boolean bIsBlinking;
00076     uint16_t u16LastToggle;
00077     uint8_t u8DutyPercent;
00078     boolean bCurrentState;
00079 } LedBlinkState_t;
00080 typedef struct {
00081     CallQueue_t strUpCalls;
00082     CallQueue_t strDownCalls;
00083     LedBlinkState_t strLedStates[cu8CALL_QUEUE_SIZE];
00084     uint16_t u16LastProcessTime;
00085 } CallHandler_t;
00086 */
00087
00089
00090 #endif // CALL_HANDLER_GCFG_H
00091
00092 /* ***** */
00093 /* ***** */
00094 /*               E N D   O F   F I L E               */
00095 /* ***** */
00096 /* ***** */

```

6.6 ElevatorController/includes/ElevatorController_Icfg.h File Reference

Local (private) configuration constants for the ElevatorController module.

Macros

- #define **DOOR_OPEN_TIME_MS** 3000
Duration the door remains open before auto-close is triggered (ms).

EEPROM Default Values

Factory-default values written to EEPROM on first power-up.

- #define **cu8SLOW_TIMER_DEF_VALUE** ((uint8_t) 7U)

- #define `cu8FAST_TIMER_DEF_VALUE` ((uint8_t) 18U)
- #define `cu8CAM_TIMER_DEF_VALUE` ((uint8_t) 3U)
- #define `cu8STOP_TIMER_DEF_VALUE` ((uint8_t) 2U)
- #define `cu8LIGHT_TIMER_DEF_VALUE` ((uint8_t) 10U)
- #define `cu8DOOR_NUMBER_DEF_VALUE` ((uint8_t) 16U)
- #define `cu8DISPLAY_DEF_VALUE` ((uint8_t) 0U)
- #define `cu8FLOOR_0_DEF_VALUE` ((uint8_t) 0U)
- #define `cu8FLOOR_1_DEF_VALUE` ((uint8_t) 1U)
- #define `cu8FLOOR_2_DEF_VALUE` ((uint8_t) 2U)
- #define `cu8FLOOR_3_DEF_VALUE` ((uint8_t) 3U)
- #define `cu8FLOOR_4_DEF_VALUE` ((uint8_t) 4U)
- #define `cu8FLOOR_5_DEF_VALUE` ((uint8_t) 5U)
- #define `cu8FLOOR_6_DEF_VALUE` ((uint8_t) 6U)
- #define `cu8FLOOR_7_DEF_VALUE` ((uint8_t) 7U)
- #define `cu8FLOOR_8_DEF_VALUE` ((uint8_t) 8U)
- #define `cu8FLOOR_9_DEF_VALUE` ((uint8_t) 9U)
- #define `cu8FLOOR_10_DEF_VALUE` ((uint8_t) 10U)
- #define `cu8FLOOR_11_DEF_VALUE` ((uint8_t) 11U)
- #define `cu8FLOOR_12_DEF_VALUE` ((uint8_t) 12U)
- #define `cu8FLOOR_13_DEF_VALUE` ((uint8_t) 13U)
- #define `cu8FLOOR_14_DEF_VALUE` ((uint8_t) 14U)
- #define `cu8FLOOR_15_DEF_VALUE` ((uint8_t) 15U)
- #define `cu8COLLECTION_DIR_DEF_VALUE` ((uint8_t) `DOWN_COLLECTIVE`)
- #define `cu8MNT_SPEED_DEF_VALUE` ((uint8_t) 0U)
- #define `cu8DOOR_OPTIONS_DEF_VALUE` ((uint8_t) 0U)
- #define `cu8PARKING_FLOOR_DEF_VALUE` ((uint8_t) 19U)
- #define `cu8CAM_FAIL_CNT_DEF_VALUE` ((uint8_t) 9U)
- #define `cu8CABIN_PW_DEF_VALUE` ((uint8_t) 0U)
- #define `cu8PHS_SEQ_DEF_VALUE` ((uint8_t) 0U)
- #define `cu8PASSWORD_L_DEF_VALUE` ((uint8_t) 0U)
- #define `cu8PASSWORD_H_DEF_VALUE` ((uint8_t) 0U)
- #define `cu8CURRENT_FLOOR_DEF_VALUE` ((uint8_t) 0U)
- #define `cu8CURRENT_TRAVEL_L_DEF_VALUE` ((uint8_t) 0U)
- #define `cu8CURRENT_TRAVEL_H_DEF_VALUE` ((uint8_t) 0U)
- #define `cu8STORED_TRAVEL_DEF_VALUE` ((uint8_t) 10U)
- #define `cu8STORED_TRAVEL_USAGE_DEF_VALUE` ((uint8_t) 0U)
- #define `cu8MASTER_PASSWORD_L_DEF_VALUE` ((uint8_t) 158U)
- #define `cu8MASTER_PASSWORD_H_DEF_VALUE` ((uint8_t) 0U)

EEPROM Minimum Values

Lower bounds enforced when saving parameters via the programming menu.

- #define `cu8SLOW_TIMER_MIN_VALUE` ((uint8_t) 1U)
- #define `cu8FAST_TIMER_MIN_VALUE` ((uint8_t) 1U)
- #define `cu8CAM_TIMER_MIN_VALUE` ((uint8_t) 1U)
- #define `cu8STOP_TIMER_MIN_VALUE` ((uint8_t) 1U)
- #define `cu8LIGHT_TIMER_MIN_VALUE` ((uint8_t) 1U)
- #define `cu8DOOR_NUMBER_MIN_VALUE` ((uint8_t) 1U)
- #define `cu8COLLECTION_DIR_MIN_VALUE` ((uint8_t) `DOWN_COLLECTIVE`)
- #define `cu8MNT_SPEED_MIN_VALUE` ((uint8_t) 0U)
- #define `cu8DOOR_OPTIONS_MIN_VALUE` ((uint8_t) 0U)
- #define `cu8PARKING_FLOOR_MIN_VALUE` ((uint8_t) 0U)
- #define `cu8CAM_FAIL_CNT_MIN_VALUE` ((uint8_t) 0U)
- #define `cu8CABIN_PW_MIN_VALUE` ((uint8_t) 0U)
- #define `cu8PHS_SEQ_MIN_VALUE` ((uint8_t) 0U)

EEPROM Maximum Values

Upper bounds enforced when saving parameters via the programming menu.

- #define `cu8SLOW_TIMER_MAX_VALUE` ((uint8_t) 20U)
- #define `cu8FAST_TIMER_MAX_VALUE` ((uint8_t) 30U)
- #define `cu8CAM_TIMER_MAX_VALUE` ((uint8_t) 20U)

- `#define cu8STOP_TIMER_MAX_VALUE ((uint8_t) 10U)`
- `#define cu8LIGHT_TIMER_MAX_VALUE ((uint8_t) 20U)`
- `#define cu8DOOR_NUMBER_MAX_VALUE ((uint8_t) 16U)`
- `#define cu8COLLECTION_DIR_MAX_VALUE ((uint8_t) 4U)`
- `#define cu8MNT_SPEED_MAX_VALUE ((uint8_t) 1U)`
- `#define cu8DOOR_OPTIONS_MAX_VALUE ((uint8_t) 1U)`
- `#define cu8PARKING_FLOOR_MAX_VALUE (cu8DOOR_NUMBER_MAX_VALUE)`
- `#define cu8CAM_FAIL_CNT_MAX_VALUE ((uint8_t) 20U)`
- `#define cu8CABIN_PW_MAX_VALUE ((uint8_t) 1U)`
- `#define cu8PHS_SEQ_MAX_VALUE ((uint8_t) 1U)`

Enumerations

- enum `CollectionDir_t` { `UP_COLLECTIVE` = 0 , `DOWN_COLLECTIVE` }

Direction used for the collective call algorithm.

6.6.1 Detailed Description

Local (private) configuration constants for the ElevatorController module.

Contains EEPROM default values, minimum values, maximum values, and internal type definitions used exclusively by ElevatorController.c and ElevatorController_lcfg.c. Must not be included by any other module.

6.7 ElevatorController_lcfg.h

[Go to the documentation of this file.](#)

```

00001 /* ***** */
00002 /* ***** */
00003 /*          (C) Copyright 2026 by          */
00004 /*          HexaMix          */
00005 /*          All rights reserved          */
00006 /*          This software is the exclusive property of HexaMix.          */
00007 /*          It cannot be communicated or divulged to anybody without a previous */
00008 /*          written authorisation.          */
00009 /*          */
00010 /*          Any partial or complete copy of this program whether it is in its */
00011 /*          original shape or in its modified shape has to mention this copyright */
00012 /*          and its proprietor.          */
00013 /*          */
00014 /* ***** */
00015 /* ***** */
00016
00017 /* ***** */
00018 /* ***** */
00019 /*          FILE INCLUSION MANAGEMENT          */
00020 /* ***** */
00021 /* ***** */
00022 /* Protection against bad and multiple inclusions          */
00023 /* ***** */
00024
00037
00038 #ifndef ELEVATOR_CONTROLLER_LCFG_H
00039 #define ELEVATOR_CONTROLLER_LCFG_H
00040
00041 /* ***** */
00042 /* Header Inclusions          */
00043 /* ***** */
00044
00045 /* other modules of the project ***** */
00046
00047
00048 #if ! ((defined ElevatorController_c) || (defined ElevatorController_lcfg_c))
00049     #error "This file must not be included from outside of it's component"
00050 #endif
00051
00052 /* ***** */

```

```

00053 /* ***** */
00054 /*          CONFIGURATION DATA          */
00055 /* ***** */
00056 /* ***** */
00057
00059 #define DOOR_OPEN_TIME_MS    3000
00060
00064 typedef enum
00065 {
00066     UP_COLLECTIVE = 0,
00067     DOWN_COLLECTIVE
00068 } CollectionDir_t;
00069
00070 /* ***** */
00071 /* ***** */
00072 /*          PERSISTENT DATA          */
00073 /* ***** */
00074 /* ***** */
00075
00080 #define cu8SLOW_TIMER_DEF_VALUE      ((uint8_t) 7U)
00081 #define cu8FAST_TIMER_DEF_VALUE      ((uint8_t) 18U)
00082 #define cu8CAM_TIMER_DEF_VALUE       ((uint8_t) 3U)
00083 #define cu8STOP_TIMER_DEF_VALUE      ((uint8_t) 2U)
00084 #define cu8LIGHT_TIMER_DEF_VALUE     ((uint8_t) 10U)
00085 #define cu8DOOR_NUMBER_DEF_VALUE     ((uint8_t) 16U)
00086 #define cu8DISPLAY_DEF_VALUE         ((uint8_t) 0U)
00087 #define cu8FLOOR_0_DEF_VALUE         ((uint8_t) 0U)
00088 #define cu8FLOOR_1_DEF_VALUE         ((uint8_t) 1U)
00089 #define cu8FLOOR_2_DEF_VALUE         ((uint8_t) 2U)
00090 #define cu8FLOOR_3_DEF_VALUE         ((uint8_t) 3U)
00091 #define cu8FLOOR_4_DEF_VALUE         ((uint8_t) 4U)
00092 #define cu8FLOOR_5_DEF_VALUE         ((uint8_t) 5U)
00093 #define cu8FLOOR_6_DEF_VALUE         ((uint8_t) 6U)
00094 #define cu8FLOOR_7_DEF_VALUE         ((uint8_t) 7U)
00095 #define cu8FLOOR_8_DEF_VALUE         ((uint8_t) 8U)
00096 #define cu8FLOOR_9_DEF_VALUE         ((uint8_t) 9U)
00097 #define cu8FLOOR_10_DEF_VALUE        ((uint8_t) 10U)
00098 #define cu8FLOOR_11_DEF_VALUE        ((uint8_t) 11U)
00099 #define cu8FLOOR_12_DEF_VALUE        ((uint8_t) 12U)
00100 #define cu8FLOOR_13_DEF_VALUE        ((uint8_t) 13U)
00101 #define cu8FLOOR_14_DEF_VALUE        ((uint8_t) 14U)
00102 #define cu8FLOOR_15_DEF_VALUE        ((uint8_t) 15U)
00103 #define cu8COLLECTION_DIR_DEF_VALUE  ((uint8_t) DOWN_COLLECTIVE)
00104 #define cu8MNT_SPEED_DEF_VALUE       ((uint8_t) 0U)
00105 #define cu8DOOR_OPTIONS_DEF_VALUE    ((uint8_t) 0U)
00106 #define cu8PARKING_FLOOR_DEF_VALUE   ((uint8_t) 19U)
00107 #define cu8CAM_FAIL_CNT_DEF_VALUE    ((uint8_t) 9U)
00108 #define cu8CABIN_PW_DEF_VALUE        ((uint8_t) 0U)
00109 #define cu8PHS_SEQ_DEF_VALUE         ((uint8_t) 0U)
00110 #define cu8PASSWORD_L_DEF_VALUE      ((uint8_t) 0U)
00111 #define cu8PASSWORD_H_DEF_VALUE      ((uint8_t) 0U)
00112 #define cu8CURRENT_FLOOR_DEF_VALUE   ((uint8_t) 0U)
00113 #define cu8CURRENT_TRAVEL_L_DEF_VALUE ((uint8_t) 0U)
00114 #define cu8CURRENT_TRAVEL_H_DEF_VALUE ((uint8_t) 0U)
00115 #define cu8STORED_TRAVEL_DEF_VALUE   ((uint8_t) 10U)
00116 #define cu8STORED_TRAVEL_USAGE_DEF_VALUE ((uint8_t) 0U)
00117 #define cu8MASTER_PASSWORD_L_DEF_VALUE ((uint8_t) 158U)
00118 #define cu8MASTER_PASSWORD_H_DEF_VALUE ((uint8_t) 0U)
00120
00125
00126
00127 /* minimum values */
00128 #define cu8SLOW_TIMER_MIN_VALUE      ((uint8_t) 1U)
00129 #define cu8FAST_TIMER_MIN_VALUE      ((uint8_t) 1U)
00130 #define cu8CAM_TIMER_MIN_VALUE       ((uint8_t) 1U)
00131 #define cu8STOP_TIMER_MIN_VALUE      ((uint8_t) 1U)
00132 #define cu8LIGHT_TIMER_MIN_VALUE     ((uint8_t) 1U)
00133 #define cu8DOOR_NUMBER_MIN_VALUE     ((uint8_t) 1U)
00134 #define cu8COLLECTION_DIR_MIN_VALUE  ((uint8_t) DOWN_COLLECTIVE)
00135 #define cu8MNT_SPEED_MIN_VALUE       ((uint8_t) 0U)
00136 #define cu8DOOR_OPTIONS_MIN_VALUE    ((uint8_t) 0U)
00137 #define cu8PARKING_FLOOR_MIN_VALUE   ((uint8_t) 0U)
00138 #define cu8CAM_FAIL_CNT_MIN_VALUE    ((uint8_t) 0U)
00139 #define cu8CABIN_PW_MIN_VALUE        ((uint8_t) 0U)
00140 #define cu8PHS_SEQ_MIN_VALUE         ((uint8_t) 0U)
00141
00146
00147 /* maximum values */
00148 #define cu8SLOW_TIMER_MAX_VALUE      ((uint8_t) 20U)
00149 #define cu8FAST_TIMER_MAX_VALUE      ((uint8_t) 30U)
00150 #define cu8CAM_TIMER_MAX_VALUE       ((uint8_t) 20U)
00151 #define cu8STOP_TIMER_MAX_VALUE      ((uint8_t) 10U)
00152 #define cu8LIGHT_TIMER_MAX_VALUE     ((uint8_t) 20U)
00153 #define cu8DOOR_NUMBER_MAX_VALUE     ((uint8_t) 16U)
00154 #define cu8COLLECTION_DIR_MAX_VALUE  ((uint8_t) 4U)
00155 #define cu8MNT_SPEED_MAX_VALUE       ((uint8_t) 1U)
00156 #define cu8DOOR_OPTIONS_MAX_VALUE    ((uint8_t) 1U)

```

```

00157 #define cu8PARKING_FLOOR_MAX_VALUE      (cu8DOOR_NUMBER_MAX_VALUE)
00158 #define cu8CAM_FAIL_CNT_MAX_VALUE        ((uint8_t) 20U)
00159 #define cu8CABIN_PW_MAX_VALUE             ((uint8_t) 1U)
00160 #define cu8PHS_SEQ_MAX_VALUE              ((uint8_t) 1U)
00161
00163
00164 #endif // ELEVATOR_CONTROLLER_LCFG_H
00165
00166 /* ***** */
00167 /* ***** */
00168 /* ***** E N D O F F I L E ***** */
00169 /* ***** */
00170 /* ***** */

```

6.8 ElevatorController/interfaces/ElevatorController.h File Reference

Public interface for the ElevatorController module.

```
#include "ElevatorController_gcfg.h"
```

```
#include <stdint.h>
```

Include dependency graph for ElevatorController.h: This graph shows which files directly or indirectly include this file:

Functions

- void **ElevatorController_Init** (void)
This function initializes and starts the operation loop of the elevator controller.
- void [ElevatorController_vidOperationLoop](#) (void)
Main periodic operation loop of the elevator controller.
- void [ElevatorController_vidProgrammingLoop](#) (void)
Programming loop of the elevator controller.
- uint8_t [ElevatorController_u8GetCurrentFloor](#) (void)
Retrieves the current floor position of the elevator.
- uint8_t [ElevatorController_u8GetSelectorCnt](#) (void)
Retrieves the selector pole count accumulated since the last reset.
- void [ElevatorController_vidStartLightTimer](#) (void)
Starts (or restarts) the cabin light auto-off timer.
- void [elevator_hal_vidInit](#) (void)
Initializes the HAL layer used by the elevator controller.
- void [elevator_hal_vidMotor_up](#) (void)
Commands the motor to run in the upward direction.
- void [elevator_hal_vidMotor_down](#) (void)
Commands the motor to run in the downward direction.
- void [elevator_hal_vidMotor_stop](#) (void)
Commands the motor to stop.
- void [elevator_hal_vidDoor_open](#) (void)
Commands the door actuator to open.
- void [elevator_hal_vidDoor_close](#) (void)
Commands the door actuator to close.
- void [elevator_hal_vidDisplay_floor](#) (uint8_t u8Floor)
Updates the floor indicator display with the given floor number.
- void [elevator_hal_vidDisplay_status](#) (const uint8_t *cpu8Status)
Updates the status display with a text string.
- void [elevator_hal_vidTimer_start](#) ()

- Starts the HAL millisecond timer.*
- void **elevator_hal_vidTimer_stop** (void)
Stops the HAL millisecond timer.
- uint16_t **elevator_hal_u16Get_time_ms** (void)
Returns the elapsed time in milliseconds since the timer was started.
- void **ElevatorController_vidSplashScreen** (void)
Displays the splash screen on the LCD at startup.
- **OperatingMode_t** **ElevatorController_enuGetMode** (void)
Returns the current operating mode of the elevator controller.

6.8.1 Detailed Description

Public interface for the ElevatorController module.

Exposes the initialization, operation loops, and service functions of the top-level elevator application controller.

6.9 ElevatorController.h

[Go to the documentation of this file.](#)

```

00001 /* ***** */
00002 /* ***** */
00003 /*          (C) Copyright 2026 by          */
00004 /*          HexaMix                        */
00005 /*          All rights reserved            */
00006 /*          This software is the exclusive property of HexaMix. */
00007 /* It cannot be communicated or divulged to anybody without a previous */
00008 /* written authorisation. */
00009 /* */
00010 /* Any partial or complete copy of this program whether it is in its */
00011 /* original shape or in its modified shape has to mention this copyright */
00012 /* and its proprietor. */
00013 /* */
00014 /* ***** */
00015 /* ***** */
00016
00017 /* ***** */
00018 /* ***** */
00019 /*          FILE INCLUSION MANAGEMENT          */
00020 /* ***** */
00021 /* ***** */
00022 /* Protection against bad and multiple inclusions */
00023 /* ***** */
00024
00025 #ifndef ELEVATOR_CONTROLLER_H
00026 #define ELEVATOR_CONTROLLER_H
00027
00028
00029
00030
00031 /* ***** */
00032 /* Header Inclusions */
00033 /* ***** */
00034
00035 /* own header inclusions ***** */
00036
00037 /* export the global configuration */
00038 #include "ElevatorController_gcfg.h"
00039 #include <stdint.h>
00040
00041
00042 /* ***** */
00043 /* ***** */
00044 /* ***** */
00045
00046 /* Basic Services ***** */
00047
00048 /* Initialise the component. */
00049 void ElevatorController_Init(void);
00050
00051

```

```

00071 void ElevatorController_vidOperationLoop(void);
00072
00080 void ElevatorController_vidProgrammingLoop(void);
00081
00082 /* Services ***** */
00083
00089 uint8_t ElevatorController_u8GetCurrentFloor(void);
00090
00099 uint8_t ElevatorController_u8GetSelectorCnt(void);
00100
00107 void ElevatorController_vidStartLightTimer(void);
00108
00115 void elevator_hal_vidInit(void);
00116
00120 void elevator_hal_vidMotor_up(void);
00121
00125 void elevator_hal_vidMotor_down(void);
00126
00130 void elevator_hal_vidMotor_stop(void);
00131
00135 void elevator_hal_vidDoor_open(void);
00136
00140 void elevator_hal_vidDoor_close(void);
00141
00147 void elevator_hal_vidDisplay_floor(uint8_t u8Floor);
00148
00154 void elevator_hal_vidDisplay_status(const uint8_t* cpu8Status);
00155
00159 void elevator_hal_vidTimer_start();
00160
00164 void elevator_hal_vidTimer_stop(void);
00165
00171 uint16_t elevator_hal_u16Get_time_ms(void);
00172
00176 void ElevatorController_vidSplashScreen(void);
00177
00183 OperatingMode_t ElevatorController_enuGetMode(void);
00184
00186
00187 /* ***** */
00188
00189 #endif // ELEVATOR_CONTROLLER_H
00190
00191 /* ***** */
00192 /* ***** */
00193 /*          E N D   O F   F I L E          */
00194 /* ***** */
00195 /* ***** */

```

6.10 ElevatorController/interfaces/ElevatorController_gcfg.h File Reference

Global configuration types for the ElevatorController module.

```

#include "Std_Types.h"
#include "SystemConfig.h"
#include "CallHandler_gcfg.h"

```

Include dependency graph for ElevatorController_gcfg.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [Elevator_t](#)

Holds the full runtime state of the elevator.

Enumerations

- enum `DoorType_t` { `DOOR_MANUAL` , `DOOR_AUTOMATIC` }
Specifies how the elevator door is controlled.
- enum `DoorState_t` { `DOOR_CLOSED` , `DOOR_OPENING` , `DOOR_OPEN` , `DOOR_CLOSING` }
Represents the current state of the elevator door.
- enum `MoveDirection_t` {
 `DIR_IDLE` , `DIR_UP` , `DIR_DOWN` , `DIR_SLOWING` ,
 `DIR_STOPPING` }
Represents the current movement direction or motion phase of the elevator.
- enum `OperatingMode_t` {
 `MODE_INIT` , `MODE_NORMAL` , `MODE_PROGRAM` , `MODE_MAINTENANCE` ,
 `MODE_ERROR` }
Represents the operating mode of the elevator controller.
- enum `Colldir_t` { `COLLECTION_UP` , `COLLECTION_DOWN` }
Specifies the collection direction for call servicing.

6.10.1 Detailed Description

Global configuration types for the ElevatorController module.

Defines all shared data types (enumerations and structs) that describe the elevator's runtime state and are exported to other modules.

6.10.2 Enumeration Type Documentation

6.10.2.1 Colldir_t

enum `Colldir_t`

Specifies the collection direction for call servicing.

Enumerator

<code>COLLECTION_UP</code>	Collect calls while travelling upward.
<code>COLLECTION_DOWN</code>	Collect calls while travelling downward.

6.10.2.2 DoorState_t

enum `DoorState_t`

Represents the current state of the elevator door.

Enumerator

<code>DOOR_CLOSED</code>	Door is fully closed.
<code>DOOR_OPENING</code>	Door is in the process of opening.

DOOR_OPEN	Door is fully open.
DOOR_CLOSING	Door is in the process of closing.

6.10.2.3 DoorType_t

enum `DoorType_t`

Specifies how the elevator door is controlled.

Enumerator

DOOR_MANUAL	Door is operated manually.
DOOR_AUTOMATIC	Door is operated automatically by the controller.

6.10.2.4 MoveDirection_t

enum `MoveDirection_t`

Represents the current movement direction or motion phase of the elevator.

Enumerator

DIR_IDLE	Elevator is stationary with no pending direction.
DIR_UP	Elevator is moving upward.
DIR_DOWN	Elevator is moving downward.
DIR_SLOWING	Elevator is decelerating before stopping.
DIR_STOPPING	Elevator is in the final stopping phase.

6.10.2.5 OperatingMode_t

enum `OperatingMode_t`

Represents the operating mode of the elevator controller.

Enumerator

MODE_INIT	Initialization mode at power-on.
MODE_NORMAL	Normal passenger service mode.
MODE_PROGRAM	Programming/configuration mode.
MODE_MAINTENANCE	Maintenance or inspection mode.
MODE_ERROR	Error / fault mode.

6.11 ElevatorController_gcfg.h

[Go to the documentation of this file.](#)

```

00001 /* ***** */
00002 /* ***** */
00003 /*          (C) Copyright 2026 by          */
00004 /*          HexaMix                      */
00005 /*          All rights reserved          */
00006 /*          This software is the exclusive property of HexaMix.          */
00007 /*          It cannot be communicated or divulged to anybody without a previous */
00008 /*          written authorisation.          */
00009 /*          */
00010 /*          Any partial or complete copy of this program whether it is in its */
00011 /*          original shape or in its modified shape has to mention this copyright */
00012 /*          and its proprietor.          */
00013 /*          */
00014 /* ***** */
00015 /* ***** */
00016
00017 /* ***** */
00018 /* ***** */
00019 /*          FILE INCLUSION MANAGEMENT          */
00020 /* ***** */
00021 /* ***** */
00022 /* Protection against bad and multiple inclusions          */
00023 /* ***** */
00024
00025 #ifndef ELEVATOR_CONTROLLER_GCFG_H
00026 #define ELEVATOR_CONTROLLER_GCFG_H
00027
00039
00040 #include "Std_Types.h"
00041 #include "SystemConfig.h"
00042 #include "CallHandler_gcfg.h"
00043
00045
00046 /* ***** */
00047 /* ***** */
00048 /* Export of CallHandler configuration settings          */
00049 /* ***** */
00050 /* ***** */
00051
00055 typedef enum
00056 {
00057     DOOR_MANUAL,
00058     DOOR_AUTOMATIC
00059 }DoorType_t;
00060
00064 typedef enum
00065 {
00066     DOOR_CLOSED,
00067     DOOR_OPENING,
00068     DOOR_OPEN,
00069     DOOR_CLOSING
00070 }DoorState_t;
00071
00075 typedef enum
00076 {
00077     DIR_IDLE,
00078     DIR_UP,
00079     DIR_DOWN,
00080     DIR_SLOWING,
00081     DIR_STOPPING
00082 }MoveDirection_t;
00083
00087 typedef enum
00088 {
00089     MODE_INIT,
00090     MODE_NORMAL,
00091     MODE_PROGRAM,
00092     MODE_MAINTENANCE,
00093     MODE_ERROR
00094 }OperatingMode_t;
00095
00099 typedef enum
00100 {
00101     COLLECTION_UP,
00102     COLLECTION_DOWN
00103 }CollDir_t;
00104
00108 typedef struct
00109 {
00110     uint8_t u8CurrentFloor;
00111     uint8_t u8DestinationFloor;
00112     uint8_t u8SlowTimer;

```

```

00113     uint8_t  u8FastTimer;
00114     uint8_t  u8CamTimer;
00115     uint8_t  u8StopTimer;
00116     uint8_t  u8LightTimer;
00117     uint8_t  u8FloorCount;
00118     uint8_t  u8MntSpeed;
00119     uint8_t  u8ParkFloor;
00120     uint8_t  u8CamFailCnt;
00121     uint8_t  u8CabinPwd;
00122     uint8_t  u8PhsSeq;
00123     uint8_t  u8CollectionDir;
00124     MoveDirection_t enuDirection;
00125     CollDir_t enuCollDir;
00126     DoorType_t enuDoorType;
00127     DoorState_t enuDoorState;
00128     OperatingMode_t enuOperatingMode;
00129     CallType_t aenuFloorCalls[cu8MAX_FLOORS];
00130     uint16_t  u16DoorTimer;
00131     boolean   bEmergencyStop;
00132 } Elevator_t;
00133
00134 #endif // ELEVATOR_CONTROLLER_GCFG_H
00135
00136 /* ***** */
00137 /* ***** */
00138 /*               E N D   O F   F I L E               */
00139 /* ***** */
00140 /* ***** */

```

6.12 LEDController_lcfg.h

```

00001 /* ***** */
00002 /* ***** */
00003 /*               (C) Copyright 2026 by               */
00004 /*               HexaMix                             */
00005 /*               All rights reserved                 */
00006 /*               This software is the exclusive property of HexaMix. */
00007 /*               It cannot be communicated or divulged to anybody without a previous */
00008 /*               written authorisation.              */
00009 /* ***** */
00010 /*               Any partial or complete copy of this program whether it is in its */
00011 /*               original shape or in its modified shape has to mention this copyright */
00012 /*               and its proprietor.                */
00013 /* ***** */
00014 /* ***** */
00015 /* ***** */
00016
00017 /* ***** */
00018 /* ***** */
00019 /*               FILE INCLUSION MANAGEMENT          */
00020 /* ***** */
00021 /* ***** */
00022 /* Protection against bad and multiple inclusions */
00023 /* ***** */
00024
00025 #ifndef LED_CONTROLLER_LCFG_H
00026 #define LED_CONTROLLER_LCFG_H
00027
00028 #if ! ((defined LEDController_c) || (defined LEDController_lcfg_c))
00029     #error "This file must not be included from outside of it's component"
00030 #endif
00031
00032 /* ***** */
00033 /* ***** */
00034 /*               CONFIGURATION DATA                */
00035 /* ***** */
00036 /* ***** */
00037
00038 /* Private defines */
00039 #define cu8SHIFT_REG_DELAY_US ((uint8_t) 1U)
00040
00041 /* Module private types */
00042 typedef struct {
00043     LEDState_t enuState;
00044     LEDPattern_t enuPattern;
00045     uint16_t u16LastToggleTime;
00046     boolean bCurrentOutput;
00047     boolean bIsInitialized;
00048 } LEDStatus_t;
00049
00050 /* LED MUX Hardware Configuration */
00051 const LEDHWConfig_t LED_HWConfig[cu8MAX_FLOORS];
00052

```

```

00053 #endif // LED_CONTROLLER_LCFG_H
00054
00055 /* ***** */
00056 /* ***** */
00057 /* ***** E N D   O F   F I L E ***** */
00058 /* ***** */
00059 /* ***** */

```

6.13 LEDController/interfaces/LEDController.h File Reference

Public interface for the LEDController module.

```

#include "Std_Types.h"
#include "dio.h"
#include "Timer.h"
#include "LEDController_gcfg.h"

```

Include dependency graph for LEDController.h:

Functions

- void [LEDController_vidInit](#) (void)
Initializes the LEDController module.
- void [LEDController_vidSetState](#) (uint8_t u8LedId, [LEDState_t](#) enuState)
Sets the instantaneous state of a specific LED.
- void [LEDController_vidTurnAllOff](#) (void)
Turns all LEDs off immediately.
- void [LEDController_vidSetPatAllOff](#) (void)
Clears the blink pattern for all LEDs.
- void [LEDController_vidSetPattern](#) (uint8_t u8LedId, [LEDPattern_t](#) enuPattern)
Assigns a blink pattern to a specific LED.
- void [LEDController_vidProcess](#) (void)
Periodic processing task for the LED controller.

6.13.1 Detailed Description

Public interface for the LEDController module.

Manages the state and blink patterns of all floor indicator LEDs connected via two 74HC573 octal latches. Provides functions to set individual LED states, assign blink patterns, and run the periodic LED processing task.

6.14 LEDController.h

[Go to the documentation of this file.](#)

```

00001 /* ***** */
00002 /* ***** */
00003 /* ***** (C) Copyright 2026 by ***** */
00004 /* ***** HexaMix ***** */
00005 /* ***** All rights reserved ***** */
00006 /* ***** This software is the exclusive property of HexaMix. ***** */
00007 /* ***** It cannot be communicated or divulged to anybody without a previous ***** */
00008 /* ***** written authorisation. ***** */
00009 /* ***** */
00010 /* ***** Any partial or complete copy of this program whether it is in its ***** */

```

```

00011 /* original shape or in its modified shape has to mention this copyright */
00012 /* and its proprietor. */
00013 /* */
00014 /* ***** */
00015 /* ***** */
00016
00017 /* ***** */
00018 /* ***** */
00019 /* FILE INCLUSION MANAGEMENT */
00020 /* ***** */
00021 /* ***** */
00022 /* Protection against bad and multiple inclusions */
00023 /* ***** */
00024
00025 #ifndef LED_CONTROLLER_H
00026 #define LED_CONTROLLER_H
00027
00041
00042 /* ***** */
00043 /* Header Inclusions */
00044 /* ***** */
00045
00046 /* cross dependency inclusions ***** */
00047 #include "Std_Types.h"
00048 #include "dio.h"
00049 #include "Timer.h"
00050
00051 /* own header inclusions ***** */
00052
00053 /* export the global configuration */
00054 #include "LEDController_gcfg.h"
00055
00056
00057 /* ***** */
00058 /* ***** */
00059 /* Exported Functions */
00060 /* ***** */
00061 /* ***** */
00062
00063 /* Basic Services ***** */
00064
00071 void LEDController_vidInit(void);
00072
00080 void LEDController_vidSetState(uint8_t u8LedId, LEDState_t enuState);
00081
00088 void LEDController_vidTurnAllOff(void);
00089
00096 void LEDController_vidSetPatAllOff(void);
00097
00107 void LEDController_vidSetPattern(uint8_t u8LedId, LEDPattern_t enuPattern);
00108
00116 void LEDController_vidProcess(void);
00117
00119
00120 #endif // LED_CONTROLLER_H
00121
00122 /* ***** */
00123 /* ***** */
00124 /* END OF FILE */
00125 /* ***** */
00126 /* ***** */

```

6.15 LEDController/interfaces/LEDController_gcfg.h File Reference

Global configuration types and hardware mappings for the LEDController module.

```

#include "std_types.h"
#include "SystemConfig.h"

```

Include dependency graph for LEDController_gcfg.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [LEDHWConfig_t](#)

Maps a logical LED index to its physical latch and bit position.

- struct [LEDConfig_t](#)

Configuration entry for a single LED pattern.

- struct [FloorLED_t](#)

Runtime state of a single floor LED.

Macros

- #define [cu8LED_CONTROLLER_PERIOD_MS](#) ((uint8_t) 10U)

Task period for the LED controller (ms).

Latch Control Pins

GPIO port and pin assignments for the two 74HC573 latch enable lines.

- #define [cenuLED_LATCH0_PORT](#) ((GPIO_t) GPIOB)
- #define [cu8LED_LATCH0_PIN](#) ((uint8_t) 2U)
- #define [cenuLED_LATCH1_PORT](#) ((GPIO_t) GPIOB)
- #define [cu8LED_LATCH1_PIN](#) ((uint8_t) 3U)

LED Data Port

8-bit parallel data port used to write LED states to the active latch.

- #define [cenuLED_DATA_PORT](#) ((GPIO_t) GPIOA)

LED Pattern Timing

On-time and off-time constants (in milliseconds) for each LED pattern.

- #define [cu16LED_INTERNAL_CALL_ON_TIME](#) ((uint16_t) 60000U)
- #define [cu16LED_INTERNAL_CALL_OFF_TIME](#) ((uint16_t) 0U)
- #define [cu16LED_EXTERNAL_CALL_ON_TIME](#) ((uint16_t) 5000U)
- #define [cu16LED_EXTERNAL_CALL_OFF_TIME](#) ((uint16_t) 5000U)
- #define [cu16LED_MAINTENANCE_ON_TIME](#) ((uint16_t) 200U)
- #define [cu16LED_MAINTENANCE_OFF_TIME](#) ((uint16_t) 200U)
- #define [cu16LED_ERROR_ON_TIME](#) ((uint16_t) 100U)
- #define [cu16LED_ERROR_OFF_TIME](#) ((uint16_t) 100U)

Enumerations

- enum [LEDCallType_t](#) { [LED_CALL_TYPE_INTERNAL](#) , [LED_CALL_TYPE_EXTERNAL](#) }

Classifies the call type associated with an LED.

- enum [LEDState_t](#) { [LED_STATE_OFF](#) , [LED_STATE_ON](#) , [LED_STATE_BLINKING](#) }

Represents the instantaneous state of a single LED.

- enum [LEDPattern_t](#) {
[LED_PATTERN_NONE](#) , [LED_PATTERN_INTERNAL_CALL](#) , [LED_PATTERN_EXTERNAL_CALL](#) ,
[LED_PATTERN_MAINTENANCE](#) ,
[LED_PATTERN_ERROR](#) }

Pre-defined blink patterns for floor indicator LEDs.

Variables

- const [LEDHWConfig_t](#) [LED_HWConfig](#) [cu8MAX_FLOORS]

External declaration of the per-floor hardware configuration array.

6.15.1 Detailed Description

Global configuration types and hardware mappings for the LEDController module.

Defines hardware pin assignments, timing constants, and the data types used to control floor indicator LEDs driven by two 74HC573 latches.

6.16 LEDController_gcfg.h

[Go to the documentation of this file.](#)

```
00001 /* ***** */
00002 /* ***** */
00003 /*          (C) Copyright 2026 by          */
00004 /*          HexaMix                      */
00005 /*          All rights reserved          */
00006 /*          This software is the exclusive property of HexaMix.          */
00007 /*          It cannot be communicated or divulged to anybody without a previous */
00008 /*          written authorisation.          */
00009 /*          */
00010 /*          Any partial or complete copy of this program whether it is in its */
00011 /*          original shape or in its modified shape has to mention this copyright */
00012 /*          and its proprietor.          */
00013 /*          */
00014 /* ***** */
00015 /* ***** */
00016
00017 /* ***** */
00018 /* ***** */
00019 /*          FILE INCLUSION MANAGEMENT          */
00020 /* ***** */
00021 /* ***** */
00022 /* Protection against bad and multiple inclusions          */
00023 /* ***** */
00024
00025 #ifndef LED_CONTROLLER_GCFG_H
00026 #define LED_CONTROLLER_GCFG_H
00027
00039
00040 #include "std_types.h"
00041 #include "SystemConfig.h"
00042
00043 /* ***** */
00044 /* ***** */
00045 /* Export of CallHandler configuration settings          */
00046 /* ***** */
00047 /* ***** */
00048
00053 #define cenuLED_LATCH0_PORT          ((GPIO_t) GPIOB)
00054 #define cu8LED_LATCH0_PIN          ((uint8_t) 2U)
00055 #define cenuLED_LATCH1_PORT          ((GPIO_t) GPIOB)
00056 #define cu8LED_LATCH1_PIN          ((uint8_t) 3U)
00058
00063 #define cenuLED_DATA_PORT          ((GPIO_t) GPIOA)
00065
00069 typedef struct {
00070     uint8_t u8LatchIndex;
00071     uint8_t u8BitPos;
00072 } LEDHWConfig_t;
00073
00078 #define cu16LED_INTERNAL_CALL_ON_TIME          ((uint16_t) 60000U)
00079 #define cu16LED_INTERNAL_CALL_OFF_TIME          ((uint16_t) 0U)
00080 #define cu16LED_EXTERNAL_CALL_ON_TIME          ((uint16_t) 5000U)
00081 #define cu16LED_EXTERNAL_CALL_OFF_TIME          ((uint16_t) 5000U)
00082 #define cu16LED_MAINTENANCE_ON_TIME          ((uint16_t) 200U)
00083 #define cu16LED_MAINTENANCE_OFF_TIME          ((uint16_t) 200U)
00084 #define cu16LED_ERROR_ON_TIME          ((uint16_t) 100U)
00085 #define cu16LED_ERROR_OFF_TIME          ((uint16_t) 100U)
00087
00089 #define cu8LED_CONTROLLER_PERIOD_MS          ((uint8_t) 10U)
00090
00094 typedef enum {
00095     LED_CALL_TYPE_INTERNAL,
00096     LED_CALL_TYPE_EXTERNAL
00097 } LEDCallType_t;
00098
00102 typedef enum {
00103     LED_STATE_OFF,
```

```

00104     LED_STATE_ON,
00105     LED_STATE_BLINKING
00106 } LEDState_t;
00107
00111 typedef enum {
00112     LED_PATTERN_NONE,
00113     LED_PATTERN_INTERNAL_CALL,
00114     LED_PATTERN_EXTERNAL_CALL,
00115     LED_PATTERN_MAINTENANCE,
00116     LED_PATTERN_ERROR
00117 } LEDPattern_t;
00118
00122 typedef struct {
00123     uint8_t u8LedId;
00124     LEDPattern_t enuPattern;
00125     uint16_t u16OnTime;
00126     uint16_t u16OffTime;
00127 } LEDConfig_t;
00128
00132 typedef struct
00133 {
00134     boolean bIsActive;
00135     LEDCallType_t enuLEDCallType;
00136     uint16_t u16BlinkTimer;
00137     boolean bLEDState;
00138 } FloorLED_t;
00139
00141 extern const LEDHWConfig_t LED_HWConfig[cu8MAX_FLOORS];
00142
00144
00145 #endif // LED_CONTROLLER_GCFG_H
00146
00147 /* ***** */
00148 /* ***** */
00149 /*          E N D   O F   F I L E          */
00150 /* ***** */
00151 /* ***** */

```

6.17 main.c File Reference

Application entry point for the HexaMix Elevator Control system.

#include "ElevatorController.h"
 Include dependency graph for main.c:

Macros

- #define F_CPU 8000000UL

Functions

- int [main](#) (void)
Application entry point.

6.17.1 Detailed Description

Application entry point for the HexaMix Elevator Control system.

Initializes the elevator controller, displays the startup splash screen, determines the boot operating mode, and dispatches to the appropriate control loop (normal operation or programming).

6.18 Menu_Icfg.h

```

00001 /* ***** */
00002 /* ***** */
00003 /*          (C) Copyright 2026 by          */
00004 /*          HexaMix                        */
00005 /*          All rights reserved            */
00006 /*          This software is the exclusive property of HexaMix. */
00007 /*          It cannot be communicated or divulged to anybody without a previous */
00008 /*          written authorisation.         */
00009 /*          */
00010 /*          Any partial or complete copy of this program whether it is in its */
00011 /*          original shape or in its modified shape has to mention this copyright */
00012 /*          and its proprietor.            */
00013 /*          */
00014 /* ***** */
00015 /* ***** */
00016
00017 /* ***** */
00018 /* ***** */
00019 /*          FILE INCLUSION MANAGEMENT      */
00020 /* ***** */
00021 /* ***** */
00022 /* Protection against bad and multiple inclusions */
00023 /* ***** */
00024
00025 #ifndef MENU_LCFG_H
00026 #define MENU_LCFG_H
00027
00028 #if ! ((defined Menu_c) || (defined Menu_lcfg_c))
00029 #error "This file must not be included from outside of it's component"
00030 #endif
00031
00032 /* ***** */
00033 /* ***** */
00034 /*          CONFIGURATION DATA            */
00035 /* ***** */
00036 /* ***** */
00037
00038 /* Menu blink hardware timer channel */
00039 #define MENU_BLINK_HW_TIMER_CHANNEL TIMER_CH2
00040
00041 #endif // MENU_LCFG_H
00042
00043 /* ***** */
00044 /* ***** */
00045 /*          E N D   O F   F I L E          */
00046 /* ***** */
00047 /* ***** */

```

6.19 Menu/interfaces/Menu.h File Reference

Public interface for the Menu module.

```
#include "Menu_gcfg.h"
```

Include dependency graph for Menu.h:

Functions

- void [Menu_Init](#) (void)
Initializes the menu system.
- void [Menu_Task](#) (void)
Periodic task handler for the menu system.
- void [Menu_Update](#) (void)
Processes user input and updates the menu state.
- void [Menu_SetItemData](#) ([MenuItemId_t](#) itemId, const char *label, uint8_t value, uint8_t minValue, uint8_t maxValue, uint8_t eepromAddress)
Configures a menu item with its label, default value, and bounds.
- void [Menu_UpdateItemValue](#) ([MenuItemId_t](#) itemId, uint8_t value)
Updates the stored value of an existing menu item.

6.19.1 Detailed Description

Public interface for the Menu module.

Provides an interactive LCD-based programming menu that allows technicians to view and modify elevator configuration parameters persisted in EEPROM. The menu is active only in [MODE_PROGRAM](#).

6.20 Menu.h

[Go to the documentation of this file.](#)

```

00001 /* ***** */
00002 /* ***** */
00003 /*          (C) Copyright 2026 by          */
00004 /*          HexaMix                        */
00005 /*          All rights reserved            */
00006 /*          This software is the exclusive property of HexaMix.          */
00007 /*          It cannot be communicated or divulged to anybody without a previous */
00008 /*          written authorisation.        */
00009 /*          */
00010 /*          Any partial or complete copy of this program whether it is in its */
00011 /*          original shape or in its modified shape has to mention this copyright */
00012 /*          and its proprietor.          */
00013 /*          */
00014 /* ***** */
00015 /* ***** */
00016
00017 /* ***** */
00018 /* ***** */
00019 /*          FILE INCLUSION MANAGEMENT          */
00020 /* ***** */
00021 /* ***** */
00022 /* Protection against bad and multiple inclusions          */
00023 /* ***** */
00024
00025 #ifndef MENU_H
00026 #define MENU_H
00027
00040
00041 /* ***** */
00042 /* Header Inclusions          */
00043 /* ***** */
00044
00045 /* cross dependency inclusions ***** */
00046
00047
00048 /* own header inclusions ***** */
00049
00050 /* export the global configuration */
00051 #include "Menu_gcfg.h"
00052
00053 #ifdef __cplusplus
00054 extern "C" {
00055 #endif
00056
00057
00058 /* ***** */
00059 /* ***** */
00060 /* Exported Functions          */
00061 /* ***** */
00062 /* ***** */
00063
00071 void Menu_Init(void);
00072
00079 void Menu_Task(void);
00080
00088 void Menu_Update(void);
00089
00100 void Menu_SetItemData(MenuItemId_t itemId, const char* label, uint8_t value, uint8_t minValue, uint8_t
    maxValue, uint8_t eepromAddress);
00101
00111 void Menu_UpdateItemValue(MenuItemId_t itemId, uint8_t value);
00112
00114
00115 #ifdef __cplusplus
00116 }
00117 #endif
00118

```

```

00119 #endif // MENU_H
00120
00121 /* ***** */
00122 /* ***** */
00123 /*          E N D   O F   F I L E          */
00124 /* ***** */
00125 /* ***** */

```

6.21 Menu/interfaces/Menu_gcfg.h File Reference

Global configuration types for the Menu module.

```
#include "std_types.h"
```

Include dependency graph for Menu_gcfg.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [MenuItem_t](#)
Data record for a single configurable menu item.

Macros

- #define **MENU_MAX_ITEMS** ((uint8_t) 10U)
Maximum number of menu items allowed in the system.
- #define **MENU_BLINK_PERIOD_MS** ((uint16_t) 20U)
Blink period for the selected menu item cursor (ms).

Enumerations

- enum [MenuItemId_t](#) {
SLOW_TIMER_MID , FAST_TIMER_MID , CAM_TIMER_MID , STOP_TIMER_MID ,
LIGHT_TIMER_MID , DOOR_NUMBER_MID , COLLECTION_DIR_MID , MNT_SPEED_MID ,
DOOR_OPTIONS_MID , PARKING_FLOOR_MID , CAM_FAIL_CNT_MID , CABIN_PW_MID ,
PHS_SEQ_MID , MENU_ITEM_COUNT }
Unique identifiers for each configurable menu item.
- enum [MenuMode_t](#) { MENU_MODE_NAVIGATE , MENU_MODE_EDIT }
Operating mode of the menu navigation state machine.

6.21.1 Detailed Description

Global configuration types for the Menu module.

Defines the menu item identifiers, operating modes, and the [MenuItem_t](#) data structure used by the programming menu system.

6.22 Menu_gcfg.h

[Go to the documentation of this file.](#)

```

00001 /* ***** */
00002 /* ***** */
00003 /*          (C) Copyright 2026 by          */
00004 /*          HexaMix                        */
00005 /*          All rights reserved            */
00006 /*          This software is the exclusive property of HexaMix.          */
00007 /*          It cannot be communicated or divulged to anybody without a previous */
00008 /*          written authorisation.        */
00009 /*          */
00010 /*          Any partial or complete copy of this program whether it is in its */
00011 /*          original shape or in its modified shape has to mention this copyright */
00012 /*          and its proprietor.          */
00013 /*          */
00014 /* ***** */
00015 /* ***** */
00016
00017 /* ***** */
00018 /* ***** */
00019 /*          FILE INCLUSION MANAGEMENT          */
00020 /* ***** */
00021 /* ***** */
00022 /* Protection against bad and multiple inclusions */
00023 /* ***** */
00024
00025 #ifndef MENU_GCFG_H
00026 #define MENU_GCFG_H
00027
00028
00029
00030 #include "std_types.h"
00031
00032
00033 /* ***** */
00034 /* ***** */
00035 /* Export of CallHandler configuration settings */
00036 /* ***** */
00037 /* ***** */
00038
00039 #define MENU_MAX_ITEMS          ((uint8_t) 10U)
00040 #define MENU_BLINK_PERIOD_MS    ((uint16_t) 20U)
00041
00042
00043 typedef enum {
00044     SLOW_TIMER_MID,
00045     FAST_TIMER_MID,
00046     CAM_TIMER_MID,
00047     STOP_TIMER_MID,
00048     LIGHT_TIMER_MID,
00049     DOOR_NUMBER_MID,
00050     COLLECTION_DIR_MID,
00051     MNT_SPEED_MID,
00052     DOOR_OPTIONS_MID,
00053     PARKING_FLOOR_MID,
00054     CAM_FAIL_CNT_MID,
00055     CABIN_PW_MID,
00056     PHS_SEQ_MID,
00057     MENU_ITEM_COUNT
00058 } MenuItemId_t;
00059
00060
00061 typedef enum {
00062     MENU_MODE_NAVIGATE,
00063     MENU_MODE_EDIT
00064 } MenuMode_t;
00065
00066
00067 typedef struct {
00068     const char* label;
00069     uint8_t value;
00070     uint8_t minValue;
00071     uint8_t maxValue;
00072     uint8_t eepromAddress;
00073 } MenuItem_t;
00074
00075
00076 #endif // MENU_GCFG_H
00077
00078
00079 /* ***** */
00080 /* ***** */
00081 /*          E N D   O F   F I L E          */
00082 /* ***** */
00083 /* ***** */
00084

```

6.23 MotionController_lcfg.h

```

00001 /* ***** */
00002 /* ***** */
00003 /*          (C) Copyright 2026 by          */
00004 /*          HexaMix                        */
00005 /*          All rights reserved            */
00006 /*          This software is the exclusive property of HexaMix.          */
00007 /*          It cannot be communicated or divulged to anybody without a previous */
00008 /*          written authorisation.        */
00009 /*          */
00010 /*          Any partial or complete copy of this program whether it is in its */
00011 /*          original shape or in its modified shape has to mention this copyright */
00012 /*          and its proprietor.          */
00013 /*          */
00014 /* ***** */
00015 /* ***** */
00016
00017 /* ***** */
00018 /* ***** */
00019 /*          FILE INCLUSION MANAGEMENT          */
00020 /* ***** */
00021 /* ***** */
00022 /* Protection against bad and multiple inclusions */
00023 /* ***** */
00024
00025 #ifndef MOTION_CONTROLLER_LCFG_H
00026 #define MOTION_CONTROLLER_LCFG_H
00027
00028 #if ! ((defined MotionController_c) || (defined MotionController_lcfg_c))
00029     #error "This file must not be included from outside of it's component"
00030 #endif
00031
00032 /* ***** */
00033 /* ***** */
00034 /*          CONFIGURATION DATA          */
00035 /* ***** */
00036 /* ***** */
00037
00038
00039 #endif // MOTION_CONTROLLER_LCFG_H
00040
00041 /* ***** */
00042 /* ***** */
00043 /*          E N D   O F   F I L E          */
00044 /* ***** */
00045 /* ***** */

```

6.24 MotionController/interfaces/MotionController.h File Reference

Public interface for the MotionController module.

```

#include "Std_Types.h"
#include "MotionController_gcfg.h"

```

Include dependency graph for MotionController.h:

Functions

- void [MotionController_vidInit](#) (void)
Initializes the MotionController module.
- void [MotionController_vidMove](#) (MotionState_t enuState)
Applies a motion command to the elevator drive.

6.24.1 Detailed Description

Public interface for the MotionController module.

Controls the elevator's drive system by activating the appropriate relay outputs for up/down high-speed and low-speed travel, as well as the controlled stop sequence.

6.25 MotionController.h

[Go to the documentation of this file.](#)

```

00001 /* ***** */
00002 /* ***** */
00003 /*          (C) Copyright 2026 by          */
00004 /*          HexaMix                        */
00005 /*          All rights reserved            */
00006 /*          This software is the exclusive property of HexaMix. */
00007 /*          It cannot be communicated or divulged to anybody without a previous */
00008 /*          written authorisation.         */
00009 /*          */
00010 /*          Any partial or complete copy of this program whether it is in its */
00011 /*          original shape or in its modified shape has to mention this copyright */
00012 /*          and its proprietor.             */
00013 /*          */
00014 /* ***** */
00015 /* ***** */
00016
00017 /* ***** */
00018 /* ***** */
00019 /*          FILE INCLUSION MANAGEMENT      */
00020 /* ***** */
00021 /* ***** */
00022 /* Protection against bad and multiple inclusions */
00023 /* ***** */
00024
00025 #ifndef MOTION_CONTROLLER_H
00026 #define MOTION_CONTROLLER_H
00027
00040
00041 /* ***** */
00042 /* Header Inclusions */
00043 /* ***** */
00044
00045 /* cross dependency inclusions ***** */
00046
00047 #include "Std_Types.h"
00048
00049 /* own header inclusions ***** */
00050
00051 /* export the global configuration */
00052 #include "MotionController_gcfg.h"
00053
00054 /* ***** */
00055 /* ***** */
00056 /* Exported Functions */
00057 /* ***** */
00058 /* ***** */
00059
00060 /* Basic Services ***** */
00067 void MotionController_vidInit(void);
00068
00079 void MotionController_vidMove(MotionState_t enuState);
00080
00082
00083 #endif // MOTION_CONTROLLER_H
00084
00085 /* ***** */
00086 /* ***** */
00087 /*          E N D   O F   F I L E          */
00088 /* ***** */
00089 /* ***** */

```

6.26 MotionController/interfaces/MotionController_gcfg.h File Reference

Global configuration types for the MotionController module.

```
#include "dio_types.h"
```

Include dependency graph for MotionController_gcfg.h: This graph shows which files directly or indirectly include this file:

Enumerations

- enum `MotionState_t` {
`MOTION_STATE_IDLE` , `MOTION_STATE_MOVING_UP_HS` , `MOTION_STATE_MOVING_UP_LS` ,
`MOTION_STATE_MOVING_DOWN_HS` ,
`MOTION_STATE_MOVING_DOWN_LS` , `MOTION_STATE_STOPPING` , `MOTION_STATE_ERROR` }

Defines the motion states for the elevator drive system.

6.26.1 Detailed Description

Global configuration types for the MotionController module.

Defines the `MotionState_t` enumeration that describes all possible motion states of the elevator drive system.

6.27 MotionController_gcfg.h

[Go to the documentation of this file.](#)

```
00001 /* ***** */
00002 /* ***** */
00003 /*          (C) Copyright 2026 by          */
00004 /*          HexaMix                      */
00005 /*          All rights reserved          */
00006 /*          This software is the exclusive property of HexaMix.          */
00007 /*          It cannot be communicated or divulged to anybody without a previous */
00008 /*          written authorisation.          */
00009 /*          */
00010 /*          Any partial or complete copy of this program whether it is in its */
00011 /*          original shape or in its modified shape has to mention this copyright */
00012 /*          and its proprietor.          */
00013 /*          */
00014 /* ***** */
00015 /* ***** */
00016
00017 /* ***** */
00018 /* ***** */
00019 /*          FILE INCLUSION MANAGEMENT          */
00020 /* ***** */
00021 /* ***** */
00022 /* Protection against bad and multiple inclusions          */
00023 /* ***** */
00024
00025 #ifndef MOTION_CONTROLLER_GCFG_H
00026 #define MOTION_CONTROLLER_GCFG_H
00027
00028
00029
00030 #include "dio_types.h"
00031
00032
00033 /* ***** */
00034 /* ***** */
00035 /* Export of CallHandler configuration settings          */
00036 /* ***** */
00037 /* ***** */
00038
00039 /* Motion States */
00040 typedef enum {
00041     MOTION_STATE_IDLE,
00042     MOTION_STATE_MOVING_UP_HS,
00043     MOTION_STATE_MOVING_UP_LS,
00044     MOTION_STATE_MOVING_DOWN_HS,
00045     MOTION_STATE_MOVING_DOWN_LS,
00046     MOTION_STATE_STOPPING,
00047     MOTION_STATE_ERROR
00048 } MotionState_t;
00049
00050
00051 #endif // MOTION_CONTROLLER_GCFG_H
00052
00053
00054 /* ***** */
00055 /* ***** */
00056 /*          E N D   O F   F I L E          */
00057 /* ***** */
00058 /* ***** */
00059 /* ***** */
```

Index

aenuFloorCalls
 Elevator_t, [34](#)
Application Layer, [7](#)

bEmergencyStop
 Elevator_t, [34](#)

blsActive
 FloorLED_t, [37](#)

bLEDState
 FloorLED_t, [37](#)

CABIN_PW_MID
 Menu Global Config, [30](#)

CALL_EXTERNAL
 CallHandler Global Config, [9](#)

CALL_INTERNAL
 CallHandler Global Config, [9](#)

CALL_NONE
 CallHandler Global Config, [9](#)

CallDioConfig_t, [33](#)

CallHandler, [7](#)
 CallHandler_vidGetCall, [8](#)
 CallHandler_vidInit, [8](#)

CallHandler Global Config, [8](#)
 CALL_EXTERNAL, [9](#)
 CALL_INTERNAL, [9](#)
 CALL_NONE, [9](#)
 CallType_t, [9](#)

CallHandler/includes/CallHandler_lcfg.h, [43](#)

CallHandler/interfaces/CallHandler.h, [44](#)

CallHandler/interfaces/CallHandler_gcfcg.h, [45](#)

CallHandler_vidGetCall
 CallHandler, [8](#)

CallHandler_vidInit
 CallHandler, [8](#)

CallType_t
 CallHandler Global Config, [9](#)

CAM_FAIL_CNT_MID
 Menu Global Config, [30](#)

CAM_TIMER_MID
 Menu Global Config, [30](#)

cenuLED_DATA_PORT
 LEDController Global Config, [24](#)

cenuLED_LATCH0_PORT
 LEDController Global Config, [24](#)

cenuLED_LATCH1_PORT
 LEDController Global Config, [24](#)

CollDir_t
 ElevatorController_gcfcg.h, [53](#)

COLLECTION_DIR_MID
 Menu Global Config, [30](#)

COLLECTION_DOWN
 ElevatorController_gcfcg.h, [53](#)

COLLECTION_UP
 ElevatorController_gcfcg.h, [53](#)

CollectionDir_t
 ElevatorController Local Config, [19](#)

cu16LED_ERROR_OFF_TIME
 LEDController Global Config, [24](#)

cu16LED_ERROR_ON_TIME
 LEDController Global Config, [24](#)

cu16LED_EXTERNAL_CALL_OFF_TIME
 LEDController Global Config, [24](#)

cu16LED_EXTERNAL_CALL_ON_TIME
 LEDController Global Config, [24](#)

cu16LED_INTERNAL_CALL_OFF_TIME
 LEDController Global Config, [24](#)

cu16LED_INTERNAL_CALL_ON_TIME
 LEDController Global Config, [25](#)

cu16LED_MAINTENANCE_OFF_TIME
 LEDController Global Config, [25](#)

cu16LED_MAINTENANCE_ON_TIME
 LEDController Global Config, [25](#)

cu8CABIN_PW_DEF_VALUE
 ElevatorController Local Config, [14](#)

cu8CAM_FAIL_CNT_DEF_VALUE
 ElevatorController Local Config, [14](#)

cu8CAM_TIMER_DEF_VALUE
 ElevatorController Local Config, [14](#)

cu8COLLECTION_DIR_DEF_VALUE
 ElevatorController Local Config, [15](#)

cu8CURRENT_FLOOR_DEF_VALUE
 ElevatorController Local Config, [15](#)

cu8CURRENT_TRAVEL_H_DEF_VALUE
 ElevatorController Local Config, [15](#)

cu8CURRENT_TRAVEL_L_DEF_VALUE
 ElevatorController Local Config, [15](#)

cu8DISPLAY_DEF_VALUE
 ElevatorController Local Config, [15](#)

cu8DOOR_NUMBER_DEF_VALUE
 ElevatorController Local Config, [15](#)

cu8DOOR_OPTIONS_DEF_VALUE
 ElevatorController Local Config, [15](#)

cu8FAST_TIMER_DEF_VALUE
 ElevatorController Local Config, [15](#)

cu8FLOOR_0_DEF_VALUE
 ElevatorController Local Config, [16](#)

cu8FLOOR_10_DEF_VALUE
 ElevatorController Local Config, [16](#)

cu8FLOOR_11_DEF_VALUE
 ElevatorController Local Config, [16](#)
 cu8FLOOR_12_DEF_VALUE
 ElevatorController Local Config, [16](#)
 cu8FLOOR_13_DEF_VALUE
 ElevatorController Local Config, [16](#)
 cu8FLOOR_14_DEF_VALUE
 ElevatorController Local Config, [16](#)
 cu8FLOOR_15_DEF_VALUE
 ElevatorController Local Config, [16](#)
 cu8FLOOR_1_DEF_VALUE
 ElevatorController Local Config, [16](#)
 cu8FLOOR_2_DEF_VALUE
 ElevatorController Local Config, [17](#)
 cu8FLOOR_3_DEF_VALUE
 ElevatorController Local Config, [17](#)
 cu8FLOOR_4_DEF_VALUE
 ElevatorController Local Config, [17](#)
 cu8FLOOR_5_DEF_VALUE
 ElevatorController Local Config, [17](#)
 cu8FLOOR_6_DEF_VALUE
 ElevatorController Local Config, [17](#)
 cu8FLOOR_7_DEF_VALUE
 ElevatorController Local Config, [17](#)
 cu8FLOOR_8_DEF_VALUE
 ElevatorController Local Config, [17](#)
 cu8FLOOR_9_DEF_VALUE
 ElevatorController Local Config, [17](#)
 cu8LED_LATCH0_PIN
 LEDController Global Config, [25](#)
 cu8LED_LATCH1_PIN
 LEDController Global Config, [25](#)
 cu8LIGHT_TIMER_DEF_VALUE
 ElevatorController Local Config, [18](#)
 cu8MASTER_PASSWORD_H_DEF_VALUE
 ElevatorController Local Config, [18](#)
 cu8MASTER_PASSWORD_L_DEF_VALUE
 ElevatorController Local Config, [18](#)
 cu8MNT_SPEED_DEF_VALUE
 ElevatorController Local Config, [18](#)
 cu8PARKING_FLOOR_DEF_VALUE
 ElevatorController Local Config, [18](#)
 cu8PASSWORD_H_DEF_VALUE
 ElevatorController Local Config, [18](#)
 cu8PASSWORD_L_DEF_VALUE
 ElevatorController Local Config, [18](#)
 cu8PHS_SEQ_DEF_VALUE
 ElevatorController Local Config, [18](#)
 cu8SLOW_TIMER_DEF_VALUE
 ElevatorController Local Config, [19](#)
 cu8STOP_TIMER_DEF_VALUE
 ElevatorController Local Config, [19](#)
 cu8STORED_TRAVEL_DEF_VALUE
 ElevatorController Local Config, [19](#)
 cu8STORED_TRAVEL_USAGE_DEF_VALUE
 ElevatorController Local Config, [19](#)
 DIR_DOWN
 ElevatorController_gcfcg.h, [54](#)
 DIR_IDLE
 ElevatorController_gcfcg.h, [54](#)
 DIR_SLOWING
 ElevatorController_gcfcg.h, [54](#)
 DIR_STOPPING
 ElevatorController_gcfcg.h, [54](#)
 DIR_UP
 ElevatorController_gcfcg.h, [54](#)
 DOOR_AUTOMATIC
 ElevatorController_gcfcg.h, [54](#)
 DOOR_CLOSED
 ElevatorController_gcfcg.h, [53](#)
 DOOR_CLOSING
 ElevatorController_gcfcg.h, [54](#)
 DOOR_MANUAL
 ElevatorController_gcfcg.h, [54](#)
 DOOR_NUMBER_MID
 Menu Global Config, [30](#)
 DOOR_OPEN
 ElevatorController_gcfcg.h, [54](#)
 DOOR_OPENING
 ElevatorController_gcfcg.h, [53](#)
 DOOR_OPTIONS_MID
 Menu Global Config, [30](#)
 DoorState_t
 ElevatorController_gcfcg.h, [53](#)
 DoorType_t
 ElevatorController_gcfcg.h, [54](#)
 DOWN_COLLECTIVE
 ElevatorController Local Config, [19](#)
 eepromAddress
 MenuItem_t, [41](#)
 elevator_hal_u16Get_time_ms
 ElevatorController, [10](#)
 elevator_hal_vidDisplay_floor
 ElevatorController, [10](#)
 elevator_hal_vidDisplay_status
 ElevatorController, [10](#)
 elevator_hal_vidInit
 ElevatorController, [11](#)
 Elevator_t, [33](#)
 aenuFloorCalls, [34](#)
 bEmergencyStop, [34](#)
 enuCollDir, [34](#)
 enuDirection, [34](#)
 enuDoorState, [35](#)
 enuDoorType, [35](#)
 enuOperatingMode, [35](#)
 u16DoorTimer, [35](#)
 u8CabinPwd, [35](#)
 u8CamFailCnt, [35](#)
 u8CamTimer, [35](#)
 u8CollectionDir, [35](#)
 u8CurrentFloor, [36](#)
 u8DestinationFloor, [36](#)
 u8FastTimer, [36](#)
 u8FloorCount, [36](#)
 u8LightTimer, [36](#)

- u8MntSpeed, 36
- u8ParkFloor, 36
- u8PhsSeq, 36
- u8SlowTimer, 37
- u8StopTimer, 37
- ElevatorController, 9
 - elevator_hal_u16Get_time_ms, 10
 - elevator_hal_vidDisplay_floor, 10
 - elevator_hal_vidDisplay_status, 10
 - elevator_hal_vidInit, 11
 - ElevatorController_enuGetMode, 11
 - ElevatorController_u8GetCurrentFloor, 11
 - ElevatorController_u8GetSelectorCnt, 11
 - ElevatorController_vidOperationLoop, 12
 - ElevatorController_vidProgrammingLoop, 12
 - ElevatorController_vidStartLightTimer, 12
- ElevatorController Global Config, 19
- ElevatorController Local Config, 13
 - CollectionDir_t, 19
 - cu8CABIN_PW_DEF_VALUE, 14
 - cu8CAM_FAIL_CNT_DEF_VALUE, 14
 - cu8CAM_TIMER_DEF_VALUE, 14
 - cu8COLLECTION_DIR_DEF_VALUE, 15
 - cu8CURRENT_FLOOR_DEF_VALUE, 15
 - cu8CURRENT_TRAVEL_H_DEF_VALUE, 15
 - cu8CURRENT_TRAVEL_L_DEF_VALUE, 15
 - cu8DISPLAY_DEF_VALUE, 15
 - cu8DOOR_NUMBER_DEF_VALUE, 15
 - cu8DOOR_OPTIONS_DEF_VALUE, 15
 - cu8FAST_TIMER_DEF_VALUE, 15
 - cu8FLOOR_0_DEF_VALUE, 16
 - cu8FLOOR_10_DEF_VALUE, 16
 - cu8FLOOR_11_DEF_VALUE, 16
 - cu8FLOOR_12_DEF_VALUE, 16
 - cu8FLOOR_13_DEF_VALUE, 16
 - cu8FLOOR_14_DEF_VALUE, 16
 - cu8FLOOR_15_DEF_VALUE, 16
 - cu8FLOOR_1_DEF_VALUE, 16
 - cu8FLOOR_2_DEF_VALUE, 17
 - cu8FLOOR_3_DEF_VALUE, 17
 - cu8FLOOR_4_DEF_VALUE, 17
 - cu8FLOOR_5_DEF_VALUE, 17
 - cu8FLOOR_6_DEF_VALUE, 17
 - cu8FLOOR_7_DEF_VALUE, 17
 - cu8FLOOR_8_DEF_VALUE, 17
 - cu8FLOOR_9_DEF_VALUE, 17
 - cu8LIGHT_TIMER_DEF_VALUE, 18
 - cu8MASTER_PASSWORD_H_DEF_VALUE, 18
 - cu8MASTER_PASSWORD_L_DEF_VALUE, 18
 - cu8MNT_SPEED_DEF_VALUE, 18
 - cu8PARKING_FLOOR_DEF_VALUE, 18
 - cu8PASSWORD_H_DEF_VALUE, 18
 - cu8PASSWORD_L_DEF_VALUE, 18
 - cu8PHS_SEQ_DEF_VALUE, 18
 - cu8SLOW_TIMER_DEF_VALUE, 19
 - cu8STOP_TIMER_DEF_VALUE, 19
 - cu8STORED_TRAVEL_DEF_VALUE, 19
 - cu8STORED_TRAVEL_USAGE_DEF_VALUE, 19
 - DOWN_COLLECTIVE, 19
 - UP_COLLECTIVE, 19
 - ElevatorController/includes/ElevatorController_lcfg.h, 46, 48
 - ElevatorController/interfaces/ElevatorController.h, 50, 51
 - ElevatorController/interfaces/ElevatorController_gcfg.h, 52, 55
 - ElevatorController_enuGetMode
 - ElevatorController, 11
 - ElevatorController_gcfg.h
 - CollDir_t, 53
 - COLLECTION_DOWN, 53
 - COLLECTION_UP, 53
 - DIR_DOWN, 54
 - DIR_IDLE, 54
 - DIR_SLOWING, 54
 - DIR_STOPPING, 54
 - DIR_UP, 54
 - DOOR_AUTOMATIC, 54
 - DOOR_CLOSED, 53
 - DOOR_CLOSING, 54
 - DOOR_MANUAL, 54
 - DOOR_OPEN, 54
 - DOOR_OPENING, 53
 - DoorState_t, 53
 - DoorType_t, 54
 - MODE_ERROR, 54
 - MODE_INIT, 54
 - MODE_MAINTENANCE, 54
 - MODE_NORMAL, 54
 - MODE_PROGRAM, 54
 - MoveDirection_t, 54
 - OperatingMode_t, 54
 - ElevatorController_u8GetCurrentFloor
 - ElevatorController, 11
 - ElevatorController_u8GetSelectorCnt
 - ElevatorController, 11
 - ElevatorController_vidOperationLoop
 - ElevatorController, 12
 - ElevatorController_vidProgrammingLoop
 - ElevatorController, 12
 - ElevatorController_vidStartLightTimer
 - ElevatorController, 12
 - enuCollDir
 - Elevator_t, 34
 - enuDirection
 - Elevator_t, 34
 - enuDoorState
 - Elevator_t, 35
 - enuDoorType
 - Elevator_t, 35
 - enuLEDCallType
 - FloorLED_t, 38
 - enuOperatingMode
 - Elevator_t, 35
 - enuPattern
 - LEDConfig_t, 38

- FAST_TIMER_MID
 - Menu Global Config, 30
- FloorLED_t, 37
 - blsActive, 37
 - bLEDState, 37
 - enuLEDCallType, 38
 - u16BlinkTimer, 38
- label
 - MenuItem_t, 41
- LED_CALL_TYPE_EXTERNAL
 - LEDController Global Config, 26
- LED_CALL_TYPE_INTERNAL
 - LEDController Global Config, 26
- LED_PATTERN_ERROR
 - LEDController Global Config, 26
- LED_PATTERN_EXTERNAL_CALL
 - LEDController Global Config, 26
- LED_PATTERN_INTERNAL_CALL
 - LEDController Global Config, 26
- LED_PATTERN_MAINTENANCE
 - LEDController Global Config, 26
- LED_PATTERN_NONE
 - LEDController Global Config, 26
- LED_STATE_BLINKING
 - LEDController Global Config, 26
- LED_STATE_OFF
 - LEDController Global Config, 26
- LED_STATE_ON
 - LEDController Global Config, 26
- LEDCallType_t
 - LEDController Global Config, 26
- LEDConfig_t, 38
 - enuPattern, 38
 - u16OffTime, 38
 - u16OnTime, 39
 - u8LedId, 39
- LEDController, 20
 - LEDController_vidInit, 20
 - LEDController_vidProcess, 20
 - LEDController_vidSetPatAllOff, 20
 - LEDController_vidSetPattern, 21
 - LEDController_vidSetState, 21
 - LEDController_vidTurnAllOff, 22
- LEDController Global Config, 22
 - cenuLED_DATA_PORT, 24
 - cenuLED_LATCH0_PORT, 24
 - cenuLED_LATCH1_PORT, 24
 - cu16LED_ERROR_OFF_TIME, 24
 - cu16LED_ERROR_ON_TIME, 24
 - cu16LED_EXTERNAL_CALL_OFF_TIME, 24
 - cu16LED_EXTERNAL_CALL_ON_TIME, 24
 - cu16LED_INTERNAL_CALL_OFF_TIME, 24
 - cu16LED_INTERNAL_CALL_ON_TIME, 25
 - cu16LED_MAINTENANCE_OFF_TIME, 25
 - cu16LED_MAINTENANCE_ON_TIME, 25
 - cu8LED_LATCH0_PIN, 25
 - cu8LED_LATCH1_PIN, 25
 - LED_CALL_TYPE_EXTERNAL, 26
 - LED_CALL_TYPE_INTERNAL, 26
 - LED_PATTERN_ERROR, 26
 - LED_PATTERN_EXTERNAL_CALL, 26
 - LED_PATTERN_INTERNAL_CALL, 26
 - LED_PATTERN_MAINTENANCE, 26
 - LED_PATTERN_NONE, 26
 - LED_STATE_BLINKING, 26
 - LED_STATE_OFF, 26
 - LED_STATE_ON, 26
 - LEDCallType_t, 26
 - LEDPattern_t, 26
 - LEDState_t, 26
- LEDController/includes/LEDController_lcfig.h, 56
- LEDController/interfaces/LEDController.h, 57
- LEDController/interfaces/LEDController_gcfig.h, 58, 60
- LEDController_vidInit
 - LEDController, 20
- LEDController_vidProcess
 - LEDController, 20
- LEDController_vidSetPatAllOff
 - LEDController, 20
- LEDController_vidSetPattern
 - LEDController, 21
- LEDController_vidSetState
 - LEDController, 21
- LEDController_vidTurnAllOff
 - LEDController, 22
- LEDHWConfig_t, 39
 - u8BitPos, 39
 - u8LatchIndex, 39
- LEDPattern_t
 - LEDController Global Config, 26
- LEDState_t
 - LEDController Global Config, 26
- LEDStatus_t, 40
- LIGHT_TIMER_MID
 - Menu Global Config, 30
- Main, 26
 - main, 27
- main
 - Main, 27
- main.c, 61
- maxValue
 - MenuItem_t, 41
- Menu, 27
 - Menu_Init, 28
 - Menu_SetItemData, 28
 - Menu_Task, 28
 - Menu_Update, 28
 - Menu_UpdateItemValue, 29
- Menu Global Config, 29
 - CABIN_PW_MID, 30
 - CAM_FAIL_CNT_MID, 30
 - CAM_TIMER_MID, 30
 - COLLECTION_DIR_MID, 30
 - DOOR_NUMBER_MID, 30
 - DOOR_OPTIONS_MID, 30
 - FAST_TIMER_MID, 30

- LIGHT_TIMER_MID, [30](#)
- MENU_ITEM_COUNT, [30](#)
- MENU_MODE_EDIT, [30](#)
- MENU_MODE_NAVIGATE, [30](#)
- MenuItemId_t, [30](#)
- MenuMode_t, [30](#)
- MNT_SPEED_MID, [30](#)
- PARKING_FLOOR_MID, [30](#)
- PHS_SEQ_MID, [30](#)
- SLOW_TIMER_MID, [30](#)
- STOP_TIMER_MID, [30](#)
- Menu/includes/Menu_lcfg.h, [62](#)
- Menu/interfaces/Menu.h, [62](#), [63](#)
- Menu/interfaces/Menu_gcfg.h, [64](#), [65](#)
- Menu_Init
 - Menu, [28](#)
- MENU_ITEM_COUNT
 - Menu Global Config, [30](#)
- MENU_MODE_EDIT
 - Menu Global Config, [30](#)
- MENU_MODE_NAVIGATE
 - Menu Global Config, [30](#)
- Menu_SetItemData
 - Menu, [28](#)
- Menu_Task
 - Menu, [28](#)
- Menu_Update
 - Menu, [28](#)
- Menu_UpdateItemValue
 - Menu, [29](#)
- MenuItem_t, [40](#)
 - eepromAddress, [41](#)
 - label, [41](#)
 - maxValue, [41](#)
 - minValue, [41](#)
 - value, [41](#)
- MenuItemId_t
 - Menu Global Config, [30](#)
- MenuMode_t
 - Menu Global Config, [30](#)
- minValue
 - MenuItem_t, [41](#)
- MNT_SPEED_MID
 - Menu Global Config, [30](#)
- MODE_ERROR
 - ElevatorController_gcfg.h, [54](#)
- MODE_INIT
 - ElevatorController_gcfg.h, [54](#)
- MODE_MAINTENANCE
 - ElevatorController_gcfg.h, [54](#)
- MODE_NORMAL
 - ElevatorController_gcfg.h, [54](#)
- MODE_PROGRAM
 - ElevatorController_gcfg.h, [54](#)
- MOTION_STATE_ERROR
 - MotionController Global Config, [32](#)
- MOTION_STATE_IDLE
 - MotionController Global Config, [32](#)
- MOTION_STATE_MOVING_DOWN_HS
 - MotionController Global Config, [32](#)
- MOTION_STATE_MOVING_DOWN_LS
 - MotionController Global Config, [32](#)
- MOTION_STATE_MOVING_UP_HS
 - MotionController Global Config, [32](#)
- MOTION_STATE_MOVING_UP_LS
 - MotionController Global Config, [32](#)
- MOTION_STATE_STOPPING
 - MotionController Global Config, [32](#)
- MotionController, [30](#)
 - MotionController_vidInit, [31](#)
 - MotionController_vidMove, [31](#)
- MotionController Global Config, [31](#)
 - MOTION_STATE_ERROR, [32](#)
 - MOTION_STATE_IDLE, [32](#)
 - MOTION_STATE_MOVING_DOWN_HS, [32](#)
 - MOTION_STATE_MOVING_DOWN_LS, [32](#)
 - MOTION_STATE_MOVING_UP_HS, [32](#)
 - MOTION_STATE_MOVING_UP_LS, [32](#)
 - MOTION_STATE_STOPPING, [32](#)
 - MotionState_t, [32](#)
- MotionController/includes/MotionController_lcfg.h, [66](#)
- MotionController/interfaces/MotionController.h, [66](#), [67](#)
- MotionController/interfaces/MotionController_gcfg.h, [67](#), [68](#)
- MotionController_vidInit
 - MotionController, [31](#)
- MotionController_vidMove
 - MotionController, [31](#)
- MotionState_t
 - MotionController Global Config, [32](#)
- MoveDirection_t
 - ElevatorController_gcfg.h, [54](#)
- OperatingMode_t
 - ElevatorController_gcfg.h, [54](#)
- PARKING_FLOOR_MID
 - Menu Global Config, [30](#)
- PHS_SEQ_MID
 - Menu Global Config, [30](#)
- SLOW_TIMER_MID
 - Menu Global Config, [30](#)
- STOP_TIMER_MID
 - Menu Global Config, [30](#)
- u16BlinkTimer
 - FloorLED_t, [38](#)
- u16DoorTimer
 - Elevator_t, [35](#)
- u16OffTime
 - LEDConfig_t, [38](#)
- u16OnTime
 - LEDConfig_t, [39](#)
- u8BitPos
 - LEDHWConfig_t, [39](#)
- u8CabinPwd

- Elevator_t, [35](#)
- u8CamFailCnt
 - Elevator_t, [35](#)
- u8CamTimer
 - Elevator_t, [35](#)
- u8CollectionDir
 - Elevator_t, [35](#)
- u8CurrentFloor
 - Elevator_t, [36](#)
- u8DestinationFloor
 - Elevator_t, [36](#)
- u8FastTimer
 - Elevator_t, [36](#)
- u8FloorCount
 - Elevator_t, [36](#)
- u8LatchIndex
 - LEDHWConfig_t, [39](#)
- u8LedId
 - LEDConfig_t, [39](#)
- u8LightTimer
 - Elevator_t, [36](#)
- u8MntSpeed
 - Elevator_t, [36](#)
- u8ParkFloor
 - Elevator_t, [36](#)
- u8PhsSeq
 - Elevator_t, [36](#)
- u8SlowTimer
 - Elevator_t, [37](#)
- u8StopTimer
 - Elevator_t, [37](#)
- UP_COLLECTIVE
 - ElevatorController Local Config, [19](#)
- value
 - MenuItem_t, [41](#)